TUGAS BESAR I

INTELEGENSI ARTIFISIAL (IF3170)

LOCAL SEARCH

Kelompok 60

More Artificial Than Intelligence

Timothy Niels Ruslim	10123053
Theodore Marvel Suranton	10623061
Qodri Azkarayan	10823501



PROGRAM STUDI TEKNIK INFORMATIKA

SEKOLAH TEKNIK ELEKTRO DAN INFORMATIKA

INSTITUT TEKNOLOGI BANDUNG

JL. GANESA 10, BANDUNG 40132

2026

DAFTAR ISI

DAFTAR ISI **2**
BAB I: DESKRIPSI PERSOALAN **3**
1.1. Latar Belakang 3
1.2. Tujuan 3
1.3. Variabel 4
1.4. Batasan 5
1.5. Fungsi Objektif 5
1.6. Ruang Pencarian 5
1.7. Asumsi 5
BAB II: PENJELASAN IMPLEMENTASI **6**
2.1. Implementasi Representasi 6
2.1.1. Package 6
2.1.2. Truck 7
2.1.3. State 7
2.1.4. Constraints 9
2.1.5. Moves 10
2.1.6. Objective 11
2.2 Implementasi Algoritma 12
2.2.1. Hill Climbing 13
2.2.2. Simulated Annealing 17
2.2.3. Genetic Algorithm 19
2.3. Implementasi Visualisasi 21
BAB III: ANALISIS DAN HASIL **23**
3.1. Hill Climbing 23
3.1.1. Steepest Ascent 23
3.1.2. Sideways-Move 26
3.1.3. Stochastic 29
3.1.4. Random Restart 39
3.2. Simulated Annealing 42
3.3. Genetic Algorithm 49
3.4. Analisis Perbandingan 61
3.4. Analisis Fungsi Objektif 68
BAB IV: KESIMPULAN DAN SARAN **70**
4.1. Kesimpulan 70
4.2. Saran 70
REFERENSI **71**
LAMPIRAN **72**

BAB I
DESKRIPSI PERSOALAN

1.1. Latar Belakang

Penyelesaian masalah merupakan salah satu kegunaan dalam intelegensi artifisial. Penyelesaian masalah merupakan proses mengidentifikasi tujuan, menganalisis opsi-opsi yang tersedia, dan membuat keputusan di situasi yang kompleks. Proses penyelesaian masalah ini terdiri atas beberapa komponen, yakni kondisi awal, aksi, model transisi, uji kondisi tujuan, dan fungsi utilitas (GeeksforGeeks, 2026).

Proses pencarian klasik seperti *depth first search* (DFS), *breadth first search* (BFS), dan A*-*search* memiliki beberapa keterbatasan, seperti penggunaan memori yang banyak dan memerlukan waktu yang lama untuk masalah yang besar dan kompleks. Oleh sebab itu, untuk permasalahan yang tidak memerlukan informasi jalur (*path-irrelevant*), dapat dilakukan pendekatan yang lebih baik, yaitu pencarian lokal. Pencarian lokal menghasilkan sebuah kondisi yang acak di awal, lalu bergerak ke kondisi tetangganya untuk mencari kondisi terbaik (Donaldson, 2019).

Terdapat bermacam-macam algoritma pencarian lokal, seperti *hill climbing*, *simulated annealing*, dan *genetic algorithm*. *Hill climbing* merupakan proses pencarian yang bergerak ke tetangga yang memiliki nilai utilitas lebih baik, menuju ke lokal optima. Sementara itu, *simulated annealing* memperbolehkan terjadinya langkah ke tetangga dengan nilai utilitas lebih rendah berdasarkan probabilitas temperatur *annealing*. Terakhir, *genetic algorithm* merupakan proses berbasis algoritma yang menerapkan operasi genetik (Donaldson, 2019). Setiap algoritma memiliki kelebihan dan kekurangannya masing-masing. Oleh karena itu, pada tugas besar ini, masing-masing algoritma dianalisis untuk mencari solusi terbaik terhadap masalah optimasi.

1.2. Tujuan

Tujuan dari tugas besar ini adalah untuk mengimplementasikan berbagai algoritma pencarian lokal dari *scratch*, lalu menganalisis dan membandingkannya. Algoritma-algoritma ini diimplementasikan untuk mendapatkan konfigurasi pengisian paket paling optimal pada sebuah truk peti kemas, dengan optimal berarti memaksimalkan fungsi objektif. Setiap algoritma dianalisis kondisi awal dan akhirnya, nilai fungsi objektif akhir yang dicapai, diagram nilai fungsi objektif terhadap banyak iterasi yang dilewati, durasi proses pencarian,

serta parameter-parameter spesifik per algoritma. Pada algoritma *hill climbing steepest ascent, stochastic,* dan *sideways move*, terdapat analisis untuk banyak iterasi hingga proses pencarian berhenti. Pada algoritma *hill climbing random restart*, terdapat tambahan analisis untuk banyak *restart* dan banyak iterasi per *restart*. Berikutnya, pada *simulated annealing*, terdapat analisis tambahan untuk plot probabilitas terhadap banyak iterasi dan frekuensi *stuck* di lokal optima. Terakhir, pada *genetic algorithm*, terdapat analisis tambahan yaitu perbandingan jumlah populasi dan banyak iterasi.

Sebagai aplikasi langsung dari algoritma *local search* ini, tugas besar ini juga bertujuan untuk menyelesaikan masalah pencarian konfigurasi pengisian paket paling untuk truk peti kemas. Ini serupa dengan masalah *knapsack* tetapi dengan unsur spasial 3D. Untuk menyelesaikan masalah ini, diperlukan representasi persoalan dan implementasi algoritma sekaligus.

1.3. Variabel

Variabel utama dalam persoalan pada tugas besar ini mengacu pada tabel di bawah ini.

Entitas	**Atribut**	**Contoh**	**Keterangan**
Package	id	“S1”	*Identifier*
	Dimensions	{w:3, l:4, h:1}	Ukuran sebuah paket, yaitu *width*, *length*, dan *height*
	Orientation	{w:x, l:y, h:z}	Sumbu yang sejajar dengan setiap sisi paket. Terdapat enam kemungkinan orientasi, misalnya {w:y, l:x, h:z} setelah rotasi 90° terhadap sumbu *z*.
	Value	60	Nilai harga sebuah paket (satuan unit)
	Weight	5	Berat paket (satuan unit berat)
	IsFragile	True	Benda pecah-belah
	ETA	3	*Estimated Time of Arrival*, perkiraan waktu kedatangan paket (dalam sisa hari)
Truck	id	“T1”	*Identifier*
	Dimensions	{w:10, l:20, h:5}	Ukuran sebuah peti kemas pada truk, yaitu *width*, *length*, dan *height*
	MaxCapacity	500	Kapasitas angkut maksimum pada suatu peti

			kemas (satuan unit berat)

Oleh karena itu, suatu *state* adalah penempatan paket-paket dalam truk-truk yang memiliki atribut seperti di atas.

1.4. Batasan

Berikut adalah batasan suatu *state* yang diperbolehkan.

1. Kardus tidak bisa melayang dan menembus dinding peti kemas. Artinya untuk setiap paket i, terdapat paket lain j yang mempunyai bagian atas tepat di bawah alas i, atau paket i berada di alas truk.
2. Barang pecah-belah tidak boleh di bawah paket lain atau menjadi support.
3. Truk tidak boleh mengangkut melebihi kapasitasnya.
4. Tidak ada tumpang tindih. Dua paket di dalam truk tidak boleh menempati daerah yang sama.

Solusi yang diberikan algoritma *local search* harus memenuhi semua batasan di atas.

1.5. Fungsi Objektif

Dalam *local search*, heuristik berupa fungsi objektif diperlukan. Pada akhirnya, tujuan utama persoalan adalah "memaksimalkan *value* peti kemas pada truk". Oleh karena itu, semua fungsi objektif yang dirancang akan ada unsur *value*. Dengan adanya batasan dan juga fungsi objektif, tugas besar ini bekerja dengan *Constraint Optimization Problem* (COP), yang membuatnya lebih kompleks daripada *Constraint Satisfaction Problem* (CSP) seperti masalah N-queens atau pewarnaan peta.

1.6. Ruang Pencarian

Untuk melakukan *local search*, pencarian tetangga akan dibatasi pada aksi berikut:

1. Menukar dua paket, baik di dalam maupun di luar truk.
2. Memindahkan sebuah paket ke suatu koordinat yang berbeda.
3. Merotasi paket 90° terhadap sumbu x, y, atau z.

Jika terdapat N paket, dengan setiap paket memiliki status di dalam atau di luar suatu truk, sehingga terdapat 2^N subset muatan. Untuk paket yang di dalam, terdapat 6 orientasi dan hingga $w \times l \times h$ posisi bilangan bulat, ditambah $N!$ kemungkinan urutan penukaran.

1.7. Asumsi

- Semua dimensi, koordinat, berat, dan nilai berupa bilangan bulat.
- Sumbu x untuk lebar, y untuk panjang, z untuk tinggi; titik asal berada di sudut kiri-bawah-belakang peti kemas.

- Tidak memperhitungkan *balancing* dalam penempatan objek diatas objek lain.

BAB II
PENJELASAN IMPLEMENTASI

Implementasi package bisa dilihat di folder `src/`. Dalam package tersebut, kami mengadopsi struktur folder berikut. Ini memastikan modularisasi kode kami tetap terstruktur dan bersih.

```
src/
├─ engine/
│  ├── package.py
│  ├── truck.py
│  ├── state.py
│  ├── constraints.py
│  ├── moves.py
│  └── objectives.py
├─ solvers/
│  ├── hill_climbing.py
│  ├── simulated_annealing.py
│  └── genetic_algorithm.py
└─ visualizer.py
```

Penjelasan khususnya akan diterangkan di subbab-subbab berikutnya.

2.1. Implementasi Representasi

Pertama, diimplementasikan representasi dari permasalahan, yang meliputi *environment*-nya (keadaan mengandung *package* dan *truck* dalam batasan) dan *objective* yang ingin dimaksimalkan agen. Di sini, diambil pendekatan *object-oriented programming* supaya implementasi modular. Implementasi dapat dilihat di `src/engine/`.

2.1.1. Package

Entitas *package* diimplementasikan sebagai kelas di file `package.py`. Detail implementasinya dirangkum dalam tabel berikut.

Kelas `Package`		
Atribut		
Nama	**Data Type**	**Deskripsi**
`id`	`string`	Identifier tunggal.
`width`	`int`	Lebar paket.

`length`	`int`	Panjang paket.
`height`	`int`	Tinggi paket.
`value`	`float`	Nilai harga paket.
`weight`	`float`	Berat paket.
`is_fragile`	`bool`	Apakah paket berisi pecah belah.
`eta`	`int`	*Estimated Time of Arrival*; perkiraan waktu kedatangan paket.

Metode			
Nama	**Parameters**	**Return**	**Deskripsi**
`get_dimensions`	`orientation`	`tuple`	Mengambil masukan suatu orientasi dan mengembalikan dimensi sebenarnya dalam orientasi tersebut

2.1.2. Truck

Entitas *truck* diimplementasikan sebagai kelas di file `truck.py`. Detail implementasinya dirangkum dalam tabel berikut.

Kelas `Truck`		
Nama	**Data Type**	**Deskripsi**
Atribut		
`id`	`string`	Identifier tunggal.
`width`	`int`	Lebar truk.
`length`	`int`	Panjang truk.
`height`	`int`	Tinggi truk.
`max_capacity`	`float`	Kapasitas berat truk.

2.1.3. State

Berikutnya, diimplementasikan representasi penempatan paket-paket dalam truk-truk pada file `state.py`. Untuk itu, dibuat kelas *placement* yang memberikan informasi mengenai

truk, koordinat, dan orientasi paket saat sudah ditempatkan. Perlu dicatat bahwa inisiasi `truck_id=None` di bawah diinterpretasikan sebagai penempatan di luar truk manapun.

Kelas `Placement`		
Nama	**Data Type**	**Deskripsi**
Atribut		
`truck_id`	`string`	Identifier truk paket ditempatkan.
`x`	`int`	Koordinat-x dalam truk paket ditempatkan.
`y`	`int`	Koordinat-y dalam truk paket ditempatkan.
`z`	`int`	Koordinat-z dalam truk paket ditempatkan.
`orientation`	`int`	Orientasi paket ditempatkan yang berupa angka 0 sampai 5 yang berkorespondensi dengan 6 orientasi berbeda suatu balok.
Metode		
`copy`	`-`	Melakukan *deep copy* kelas `Placement`.

Dengan adanya cara untuk merepresentasikan penempatan paket, dapat dibuat kelas untuk mewakili keadaan *environment*, yaitu kelas `State` sebagai berikut.

Kelas `State`			
Nama	**Data Type**		**Deskripsi**
Atribut			
`packages`	`list[Packages]`		Daftar semua paket.
`trucks`	`list[Trucks]`		Daftar semua truk.
`placements`	`dict`		*Dictionary* dengan *key* berupa paket dan *value* adalah suatu `Placement` yang menyimpan informasi penempatan paket tersebut di *state* ini.
Metode			
Nama	**Parameters**	**Return**	**Deskripsi**
`initialize_placements`	`string`	`-`	Prosedur yang

			menginisialisasikan atribut `placements` berdasarkan tiga mode: *outside*, *random*, atau *random feasible*.
`get_placement`	`package_id`	`Placement`	Fungsi yang mengembalikan penempatan dari suatu paket di state tersebut.
`set_placement`	`package_id, placement`	-	Prosedur yang menggantikan nilai atribut `placements` pada indeks `package_id` dengan `placement`.
`get_packages_in_truck`	`truck_id`	`list[Packages]`	Fungsi yang mengembalikan daftar semua paket yang ditempatkan pada truk dengan *identifier* `truck_id`.
`get_packages_outside`	-	`list[Packages]`	Fungsi yang mengembalikan daftar semua paket dengan penempatan `truck_id=None`.
`copy`	-		Melakukan *deep copy* kelas `State`.

2.1.4. Constraints

Berikutnya, setelah representasi keadaan *environment* sekarang sudah selesai, bisa diimplementasikan batasan yang berlaku supaya mengetahui apakah suatu keadaan *valid*. Untuk itu, kami memutuskan untuk membuat fungsi-fungsi terpisah dari kelas `State` yang bertugas untuk mengecek validitas suatu *state*. Berikut adalah fungsi-fungsi yang dibuat dalam file `constraints.py`.

Fungsi `constraints`			
Metode			
Nama	**Parameters**	**Return**	**Deskripsi**
`get_bounding_box`	`package, placement`	`tuple[int]`	Mencari *bounding box* suatu paket diberikan penempatannya, yaitu kedua koordinat ekstrimnya.
`check_capacity`	`packages_in_truck, truck`	`bool`	Mengecek apakah total berat paket-paket dalam suatu truk

			melampaui kapasitas truk.
`check_boundary`	`packages_in_truck, truck`	`bool`	Mengecek apakah ada paket dalam suatu truk yang *bounding box*-nya di luar kotak truk.
`check_overlap`	`packages_in_truck`	`bool`	Mengecek apakah terdapat kotak-kotak yang saling *overlap*.
`check_support`	`packages_in_truck`	`bool`	Mengecek apakah suatu kotak memiliki *support* (di bawahnya ada paket lain atau dasar truk).
`check_fragile`	`packages_in_truck`	`bool`	Mengecek apakah suatu paket pecah belah ditempatkan di bawah paket tak pecah-belah.
`is_valid`	`state`	`bool`	Memanggil `check_capacity`, `check_boundary`, `check_overlap`, `check_support`, dan `check_fragile` untuk setiap paket dan truk di suatu *state*.

2.1.5. Moves

Karena bekerja dengan *local search*, diperlukan cara untuk berpindah dari suatu *state* ke *state* tetangga. Berdasarkan yang telah didiskusikan sebelumnya, ini hanya melibatkan tiga hal: menukar penempatan dua paket, memindahkan satu paket, dan merotasikan paket. Ketiga hal ini dapat dicoba untuk mendapatkan suatu tetangga acak atau (dengan *brute force*) semua tetangga yang mungkin. Ini diimplementasikan di `moves.py`.

Fungsi `moves`			
Metode			
Nama	**Parameters**	**Return**	**Deskripsi**
`swap_packages`	`state, package_id_a, package_id_b`	`State`	Menukar penempatan `package_id_a` dan `package_id_b` di *state* mereka.
`move_package`	`state, package_id, new_placement`	`State`	Menempatkan `package_id` sesuai `new_placement`.
`rotate_package`	`state, package_id,`	`State`	Merotasikan `package_id` sesuai orientasi `new_orientation`.

	`new_orientation`		
`get_random_neighbor`	`state`	`State`	Mengembalikan suatu *state* tetangga secara acak dengan melakukan salah satu dari ketiga aksi di atas.
`get_all_neighbors`	`state`	`list[State]`	Mengembalikan daftar semua *state* tetangga yang mungkin dengan melakukan *brute force*.

2.1.6. Objective

Akhirnya, untuk menyiapkan algoritma *local search*, diperlukan fungsi objektif yang menilai suatu *state*. Untuk ini, implementasi berbasis kelas lagi supaya memungkinkan implementasi banyak jenis fungsi objektif yang masih mengikuti kerangka umum.

Kelas `ObjectiveFunction`			
Atribut			
Nama	**Data Type**		**Deskripsi**
`penalty`	`float`		Penalti untuk *state* yang tidak valid (tidak memenuhi batasan).
Metode			
Nama	**Parameters**	**Return**	**Deskripsi**
`evaluate`	`state`	`float`	Menghitung nilai objektif suatu *state* berdasarkan rumus tertentu.

Di sini, kita definisikan tiga jenis objektif, yang berarti tiga kelas yang meng-*inherit* kelas `ObjectiveFunction`. Berikut ringkasan mereka.

Objective Function	Formula	Deskripsi
`MaxValueObjective`	$\sum_{t \in \text{trucks}} \sum_{p \in \text{packages}} p.\text{value}$	Ini objektif *default* (sesuai spesifikasi) yang hanya menghitung total nilai semua paket yang dimasukan ke dalam truk.

`TimeAwareObjective`	$\sum_{t \in \text{trucks}} \sum_{p \in \text{packages}} \frac{p.\text{value}}{1 + p.\text{eta}}$	Ini objektif yang memperhitungkan ETA paket, sehingga *local search* akan memprioritaskan penempatan paket dengan ETA kecil. Di sini, penjumlahan dengan 1 di bawah supaya tidak membagi dengan 0 saat ETA adalah 0.
`GravityAwareObjective`	$\sum_{t \in \text{trucks}} \sum_{p \in \text{packages}} p.\text{value} \left(1 - \frac{p.\text{z} + p.\text{height}/2}{t.\text{height}}\right)$	Ini objektif yang memperhitungkan dimensi setiap paket relatif truk yang ditempatkannya. Dihitungkan rasio ketinggian (*centroid* paket terhadap tinggi truk) sebagai acuan. Dengan ini, penempatan paket yang besar diprioritaskan di bawah. Ini menjelaskan nama *gravity awareness*.

2.2 Implementasi Algoritma

Selanjutnya, dengan implementasi representasi *environment* dan landasan manipulasinya sudah selesai, dapat diimplementasikan algoritma *local search*-nya. Implementasi dapat dilihat di `src/solvers/`. Sebelum membuat setiap algoritmanya satu-satu, dibuat terlebih dahulu penyetaraan implementasi dan output mereka. Untuk itu, pertama dibuat kelas `Result` sebagai berikut.

Kelas `Result`		
Atribut		
Nama	**Data Type**	**Deskripsi**
`solver`	`string`	Nama algoritma yang digunakan (bukan kelas `Solver`).
`objective`	`string`	Nama fungsi objektif yang digunakan (bukan kelas ObjectiveFunction).
`initial_state`	`State`	*State* awal.
`final_state`	`State`	*State* akhir yang dihasilkan oleh algoritma.
`objective_history`	`list[float]`	Larik yang menyimpan sejarah nilai objektif setiap *state* yang ditelusuri

		oleh algoritma.
`total_iterations`	`int`	Iterasi total yang ditempuh.
`runtime`	`float`	Waktu komputasi algoritma (dalam satuan detik).
`metadata`	`dict`	*Dictionary* yang berisi *metadata* lainnya yang hanya berlaku untuk algoritma khusus (contohnya *setting* jumlah populasi pada *genetic algorithm*).

Metode			
Nama	**Parameters**	**Return**	**Deskripsi**
`__str__`	`-`	`-`	Menampilkan hasil *result* dengan terformatkan supaya bisa membaca hasil suatu algoritma dengan mudah.

Dengan ini, diimplementasikan kelas `Solver` yang akan di-*inherit* semua algoritma yang akan dibuat berikutnya untuk menjaga modularitas.

Kelas `Solver`		
Atribut		
Nama	**Data Type**	**Deskripsi**
`objective`	`ObjectiveFunction`	Fungsi objektif yang digunakan.

Metode			
Nama	**Parameters**	**Return**	**Deskripsi**
`solve`	`initial_state`	`Result`	Mengambil suatu *state* awal, melakukan *local search* dengan algoritma yang bersesuaian, lalu mengembalikan hasilnya dalam bentuk kelas `Result` (lihat atas).

2.2.1. Hill Climbing

Implementasi *hill climbing* dilakukan mengikuti pseudocode pada Russell (2021). Algoritma *hill climbing* dibagi menjadi empat algoritma lanjutan, yakni *steepest ascent*, *sideways move*, *stochastic*, dan *random restart*. Algoritma ini meng-*inherit* dari `Solver`. Berikut detailnya untuk *parent class* hill climbing.

Kelas `HillClimbing`		
Atribut		
Nama	**Data Type**	**Deskripsi**
`objective`	`ObjectiveFunction`	Kelas fungsi objektif yang digunakan.
Kelas `SteepestAscent`		
Atribut		
Nama	**Data Type**	**Deskripsi**
`objective`	`ObjectiveFunction`	Kelas fungsi objektif yang digunakan.
`max_iterations`	`int`	Iterasi maksimum (untuk kriteria pemberhentian lebih awal).
Metode		
Name	Pseudocode	

`solve`	`solve(initial_state)`
	`  current ← initial_state`
	`  for t = 1 to max_iterations do`
	`  successor ← get_all_neighbor(current)`
	`  neighbor ← highest successor`
	`  if objective(neighbor) > objective(current)`
	`  then return current ← neighbor`
	`  return current`	

Kelas `SidewaysMove`		
Atribut		
Nama	**Data Type**	**Deskripsi**
`objective`	`ObjectiveFunction`	Kelas fungsi objektif yang digunakan.
`max_sideways_moves`	`int`	Jumlah gerakan menyamping yang diperbolehkan.
`max_iterations`	`int`	Iterasi maksimum (untuk kriteria pemberhentian lebih awal).
Metode		
Name	Pseudocode	

solve	**solve**(initial_state) current ← initial_state **for** t = 1 **to** max_iterations **do** successor ← **get_all_neighbor**(current) neighbor ← highest successor **if objective**(neighbor) > **objective**(current) **then return** current ← neighbor **elif objective**(neighbor) = **objective**(current) **and** sideways_move < max_sideways_move **then return** current ← neighbor sideways_move ← +1 **return** current

Kelas `Stochastic`

Atribut

Nama	Data Type	Deskripsi
objective	ObjectiveFunction	Kelas fungsi objektif yang digunakan.
max_random_successor_iterations	int	Iterasi maksimum pembangkitan random successor.
max_iterations	int	Iterasi maksimum (untuk kriteria pemberhentian lebih awal).

Metode

Name	Pseudocode
solve	**solve**(initial_state) current ← initial_state **for** t = 1 **to** max_iterations **do** successor ← **get_random_neighbor**(current) **if objective**(neighbor) > **objective**(current) **then return** current ← neighbor random_successor_iterations ← 0 **else then** random_successor_iterations ← +1 **if** random_successor_iterations ≥ max_random_successor_iterations **break** **return** current

Kelas `RandomRestart`

Atribut

Nama	Data Type	Deskripsi
objective	ObjectiveFunction	Kelas fungsi objektif yang digunakan.

`max_restarts`	`int`	Jumlah restart maksimum yang dilakukan.
`max_iterations_per_restart`	`int`	Iterasi maksimum per restart (untuk kriteria pemberhentian lebih awal).
Metode		
Name	Pseudocode	

`solve`	`solve(initial_state)`
	`  start_state ← initial_state`
	`  for restart = 1 to max_restarts do`
	`  current ← start_state`
	`  for t = 1 to max_iterations do`
	`  successor ← get_all_neighbor(current)`
	`  neighbor ← highest successor`
	`  if objective(neighbor) > objective(current)`
	`  then return current ← neighbor`
	`  return current`
	`  start_state ← randomize(initial_state)`	

2.2.2. Simulated Annealing

Implementasi *simulated annealing* akan mengikuti pseudocode yang ada di Russell (2021). Untuk itu, diperlukan terlebih dahulu suatu schedule yang menyatakan temperatur sebagai fungsi dari iterasi. Untuk itu, diimplementasikan kelas Schedule sebagai berikut.

Kelas `Schedule`			
Atribut			
Nama	**Data Type**		**Deskripsi**
`initial_temperature`	`float`		Nilai temperature awal (saat iterasi pertama).
Metode			
Nama	**Parameters**	**Return**	**Deskripsi**
`__call__`	`t`	`float`	Menghitung temperatur pada iterasi ke-`t`.

Lalu, akan diimplementasikan beberapa *schedule* dari daftar lengkap pada Lemoine (2026). Dipilih empat yang paling utama sebagai berikut.

Schedule	Formula	Deskripsi

`GeometricSchedule`	$S(t) = T_0 \alpha^t$	Ini adalah barisan geometri dengan rasio $0 < \alpha < 1$.
`LinearSchedule`	$S(t) = T_0 - \beta t$	Ini adalah barisan aritmetika dengan selisih $-\ \beta$.
`RationalSchedule`	$S(t) = \frac{T_0}{1 + \beta T_0 t}$	Ini adalah *schedule* dengan fungsi rasional yang dibuat oleh Lundy & Mees dengan parameter β.
`LogarithmicSchedule`	$S(t) = T_0 \frac{\ln(2)}{\ln(t+2)}$	Ini adalah *schedule* dengan fungsi logaritma yang dibuat oleh Hajek. Secara teoritis, *schedule* ini dapat memastikan *simulated annealing* mengeksplorasi *state-state* yang mungkin dengan baik, walaupun secara praktis sangat lambat.

Akhirnya, algoritma *simulated annealing* sendiri meng-*inherit* dari `Solver`. Berikut detailnya dan *pseudocode* fungsi `solve`-nya.

Kelas `SimulatedAnnealing`		
Atribut		
Nama	**Data Type**	**Deskripsi**
`objective`	`ObjectiveFunction`	Kelas fungsi objektif yang digunakan.
`schedule`	`Schedule`	Kelas *schedule* yang digunakan (lihat di atas).
`minimum_temperature`	`float`	Suhu minimum (untuk kriteria pemberhentian lebih awal).
`max_iterations`	`int`	Iterasi maksimum (untuk kriteria pemberhentian lebih awal).
Metode		
Name	Pseudocode	

`solve`	`solve(initial_state)`
	`  current ← initial_state`
	`  for t = 1 to ∞ do`
	`  T ← schedule(t)`
	`  if T ≤ 0 then`
	`  return current`
	`  next ← get_random_neighbor(current)`
	`  ΔE ← objective(current) − objective(next)`	

	p = **min**(1, exp(ΔE/T)) current ← next **with probability** p

Kelas `SimulatedAnnealing` akan mengembalikan suatu `Result` dengan informasi tambahan mengenai sejarah nilai peluang `p` dan banyak *local optimum* yang tercapai. Untuk menghitung yang kedua, digunakan fungsi pembantu berikut.

Fungsi `count_local_optimum`		
Parameters	**Return**	**Deskripsi**
`arr`	`int`	Menghitung jumlah *local optimum* pada suatu larik `arr`. Definisi *local optimum* kami dilihat dari sejarah nilai objektif selama penjalanan algoritma. Suatu entri dikatakan *local optimum* jika terdapat lingkungan di mana entri tersebut yang terbesar. Secara algoritma, dihitung dengan menghilangkan duplikat bersebelahan pada `arr`, lalu menghitung banyak entri yang nilainya lebih besar dari entri kiri dan kanannya.

2.2.3. Genetic Algorithm

Implementasi Genetic Algorithm mengikuti salindia perkuliahan IF3170 tahun 2026 dengan mengikuti struktur kode dari github Priambodo (2025) dan textbook Goldberg (1989).

Kelas `GeneticAlgorithm`		
Atribut		
Nama	**Data Type**	**Deskripsi**
`objective`	`ObjectiveFunction`	Kelas fungsi objektif yang digunakan.
`population_size`	`int`	Jumlah populasi yang dibangkitkan di satu generasi.
`max_generations`	`int`	Iterasi maksimum (untuk kriteria pemberhentian).
Metode		
Name	Pseudocode	
`create_population`	`FUNCTION CreatePopulation(initial_state, population_size)` `    population ← list kosong`	

	REPEAT population_size TIMES state ← State baru dengan init = "random_feasible" tambahkan state ke population RETURN population
select	FUNCTION Select(population, scores) finite ← semua skor yang bukan ±inf/nan IF finite kosong THEN RETURN individu acak dari population min_score ← MIN(finite) FOR setiap score di scores IF score finite THEN weight ← (score - min_score) + 1e-6 ELSE weight ← 0 RETURN satu individu dari population, dipilih acak dengan peluang proporsional terhadap weight
crossover	FUNCTION Crossover(parent_a, parent_b) packages ← daftar paket IF jumlah packages ≤ 1 THEN RETURN copy(parent_a) child ← State kosong point ← bilangan bulat acak dalam [1, jumlah packages - 1] FOR i, package di packages IF i < point THEN parent ← parent_a ELSE parent ← parent_b placement ← placement package tsb di parent IF placement.truck_id = None THEN LANJUT ke paket berikutnya pasang copy(placement) ke package di child IF child tidak valid THEN kembalikan package ke luar truk RETURN child
mutate	FUNCTION Mutate(state, max_attempts = 10) REPEAT max_attempts KALI neighbor ← RandomNeighbor(state) IF neighbor valid THEN RETURN neighbor RETURN state
solve	FUNCTION Solve(initial_state, population_size, max_generations) population ← CreatePopulation(initial_state, population_size) scores ← [Evaluate(s) UNTUK setiap s di population] best_state ← individu dengan skor tertinggi best_objective ← skor tertinggi objective_history ← [best_objective]

	max_history ← [MAX(scores)] average_history ← [Average(scores)] mulai timer REPEAT max_generations TIMES new_population ← list kosong WHILE jumlah new_population < population_size parent_a ← Select(population, scores) parent_b ← Select(population, scores) child ← Crossover(parent_a, parent_b) child ← Mutate(child) tambahkan child ke new_population population ← new_population scores ← [Evaluate(s) UNTUK setiap s di population] generation_best ← MAX(scores) IF generation_best > best_objective THEN best_objective ← generation_best best_state ← copy tambahkan best_objective ke objective_history tambahkan generation_best ke max_history tambahkan Average(scores) ke average_history hentikan timer RETURN Result(initial_state, best_state, objective_history, max_history, average_history, max_generations, durasi, population_size, best_objective)

2.3. Implementasi Visualisasi

Terakhir, di sini diimplementasikan cara memvisualisasikan suatu *state*. Untuk itu, digunakan package `Plotly`, karena bisa memberikan visual 3D yang interaktif dengan sangat mudah dalam suatu *notebook* Python. Implementasi ini bisa dilihat di `src/visualizer.py`. Di sana, diimplementasikan banyak *helper functions* untuk melakukan visualisasi, walaupun fungsi utama adalah fungsi `draw_state`. Berikut detailnya.

Fungsi `visualizer`			
Metode			
Nama	**Parameters**	**Return**	**Deskripsi**
`draw_package`	`package, x0, y0, z0, dx, dy, dz, origin, color, opacity, name`	`Mesh3D`	Fungsi pembantu yang memberikan yang akan menggambarkan suatu paket dengan koordinat, dimensi, dan

			warna tertentu. Implementasinya dengan *mesh* supaya mudah membuat setiap muka paket.
`draw_truck`	`truck, origin, color, name`	`Scatter3D`	Fungsi pembantu yang menggambarkan *outline* suatu truk (sebagai *wireframe*).
`draw_unassigned_area`	`width, length, origin, name`	`Scatter3D`	Fungsi pembantu yang menggambarkan *outline* daerah untuk menaruh paket-paket yang tidak dalam truk.
`draw_state`	`state, truck_gap`	`-`	Fungsi utama yang memvisualisasikan suatu *state* seutuhnya dengan memanggil ketiga fungsi di atas untuk setiap paket pada *state*.

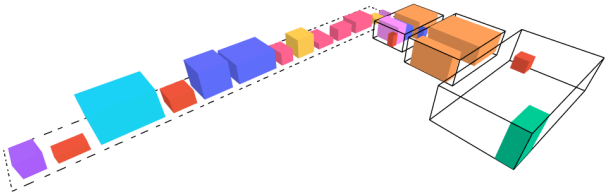
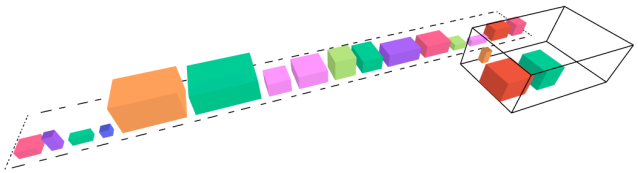
BAB III
ANALISIS DAN HASIL

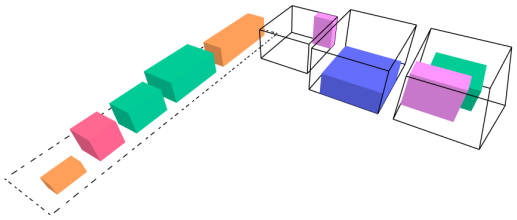
3.1. Hill Climbing

Berikut adalah hasil dari algoritma *hill climbing*, terdiri atas *steepest ascent*, *sideways move*, *stochastic*, dan *random restart*.

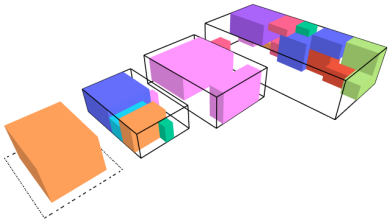
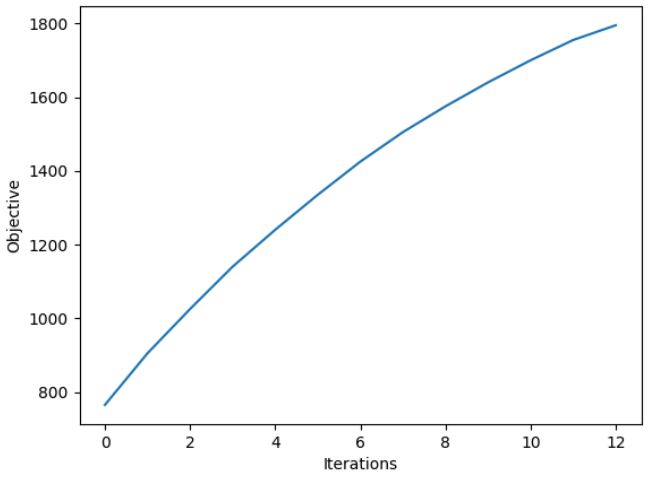
3.1.1. Steepest Ascent

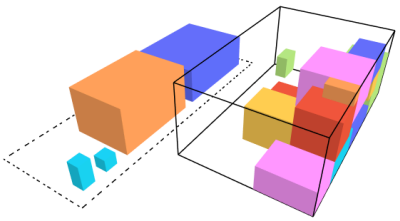
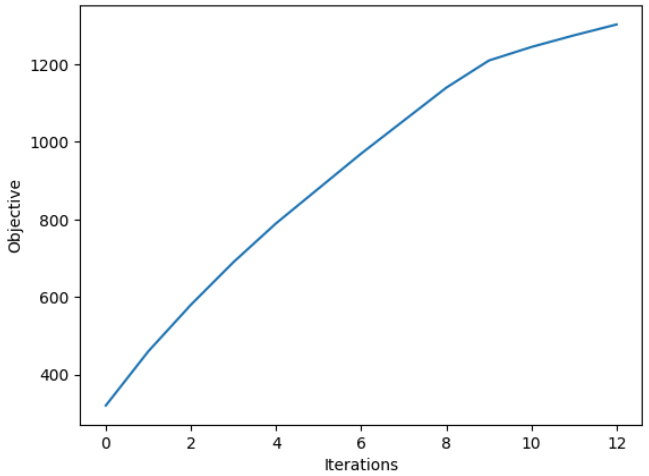
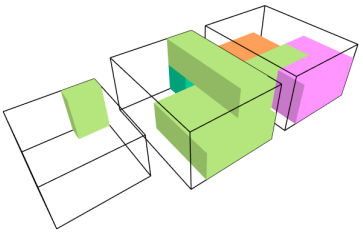
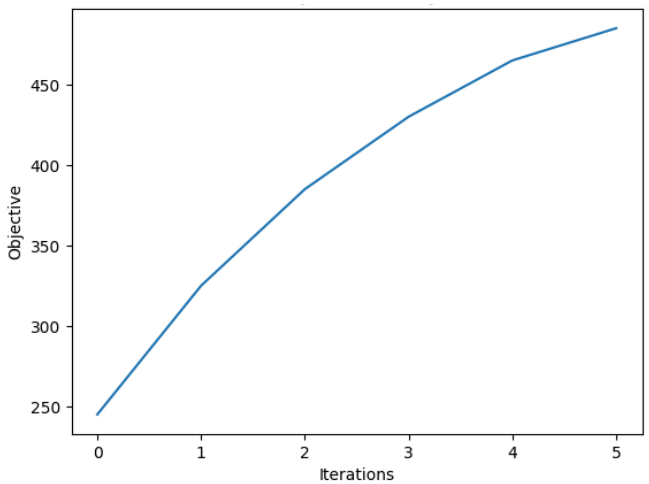
Eksperimen algoritma *steepest ascent hill climbing* dilakukan pada tiga dataset beragam berikut (yang menguji kombinasi banyak truk dan paket yang berbeda).

Dataset	Information	Value
1	Truck Count	3
	Package Count	23
	Initial Objective	765
	Visualization	
2	Truck Count	1
	Package Count	19
	Initial Objective	320
	Visualization	
3	Truck Count	3
	Package Count	9
	Initial Objective	245

	Visualization	

Algoritma di-*run* satu kali pada setiap *dataset* (satu kali cukup karena *steepest ascent* merupakan algoritma deterministik). Hasilnya dirangkum pada tabel berikut.

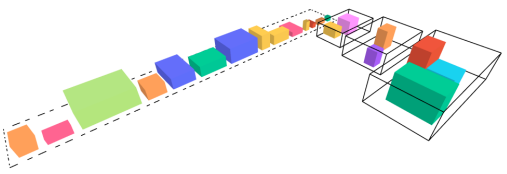
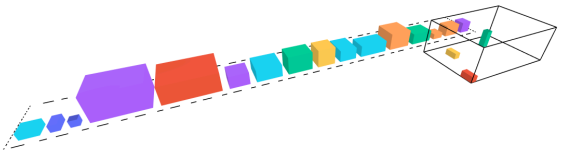
Dataset	**Information**	**Value**
1	Runtime	77.521 s
	Total Iterations	13
	Final Objective	1795
	Visualization	
	Objective History	
2	Runtime	46.734 s
	Total Iterations	13
	Final Objective	1303

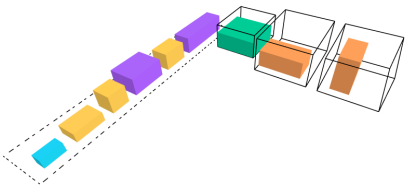
	Visualization	
	Objective History	
3	Runtime	1.153 s
	Total Iterations	6
	Final Objective	485
	Visualization	
	Objective History	

Algoritma *steepest ascent hill climbing* merupakan algoritma *hill climbing* yang relatif sederhana dibanding algoritma *hill climbing* lainnya, sebab algoritma ini tidak dapat melakukan gerakan menyamping ataupun turun. Algoritma ini sangat efektif sebab langsung menuju *state* tertinggi dari *neighbor state*. Oleh karena algoritma ini membangkitkan seluruh *neighbor state* dari *current state*, algoritma ini memiliki durasi pencarian (*runtime*) yang moderat, dengan durasi tergantung pada kompleksitas dataset. Kurva objektif mendekati linear dan mampu mencapai local optima dengan jumlah iterasi yang relatif sedikit. Penambahan repetisi *running* algoritma tidak akan mengubah hasil pencarian, sebab algoritma ini akan selalu melalui jalur yang sama karena hanya menggunakan *highest successor*. Oleh karena itu, selama *initial state* masih sama, maka jalur yang ditempuh juga akan selalu sama.

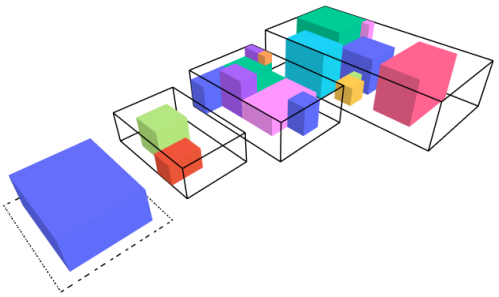
3.1.2. Sideways-Move

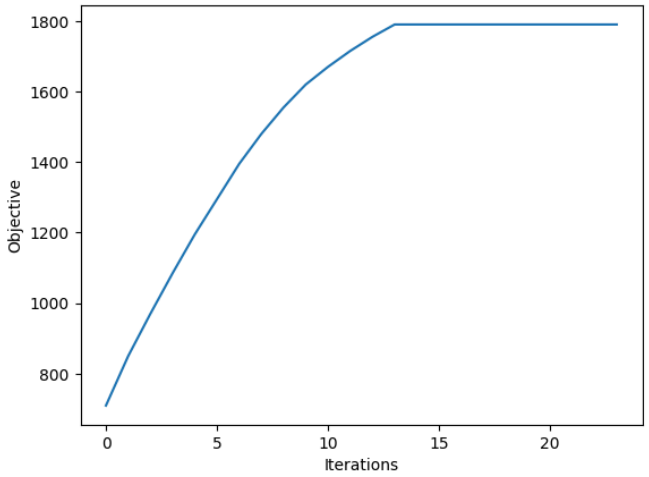
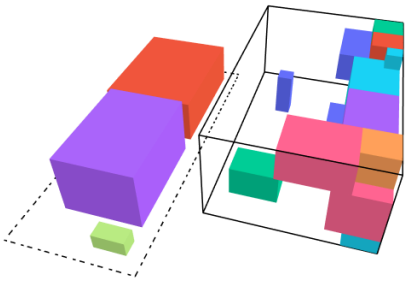
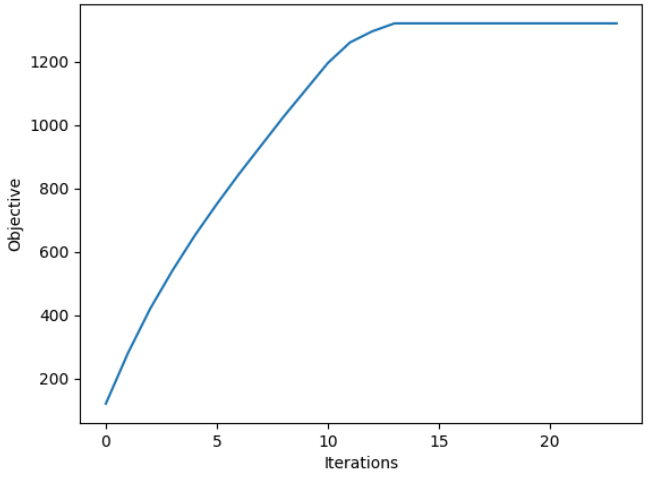
Eksperimen algoritma *sideways move hill climbing* dilakukan pada tiga dataset beragam berikut (yang menguji kombinasi banyak truk dan paket yang berbeda).

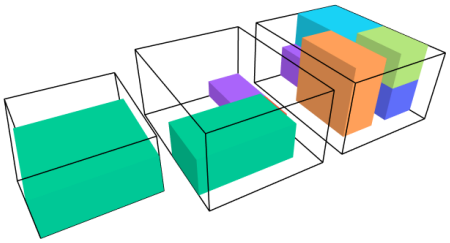
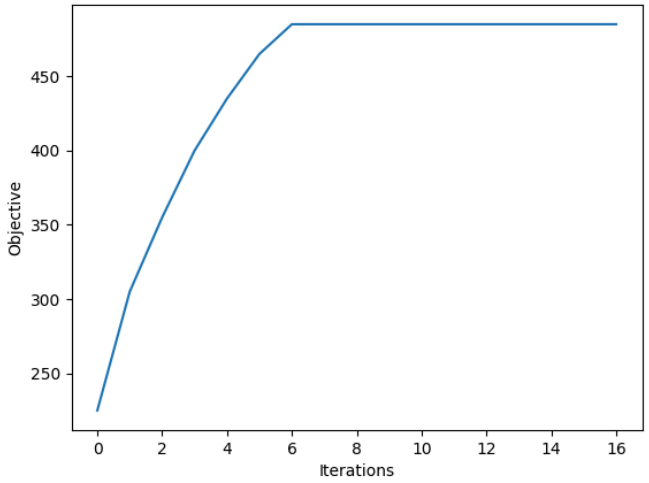
Dataset	Information	Value
1	Truck Count	3
	Package Count	23
	Initial Objective	710
	Visualization	
2	Truck Count	1
	Package Count	19
	Initial Objective	120
	Visualization	

3	Truck Count	3
	Package Count	9
	Initial Objective	225
	Visualization	

Algoritma di-*run* satu kali pada setiap *dataset* (satu kali cukup karena *sideways move* berupa algoritma deterministik). Jumlah maksimum *sideways move* untuk semua dataset adalah 10. Hasilnya dirangkum pada tabel berikut.

Dataset	**Information**	**Value**
1	Runtime	142.342 s
	Total Iterations	24
	Final Objective	1790
	Visualization	

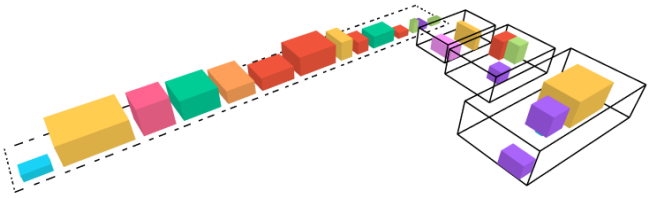
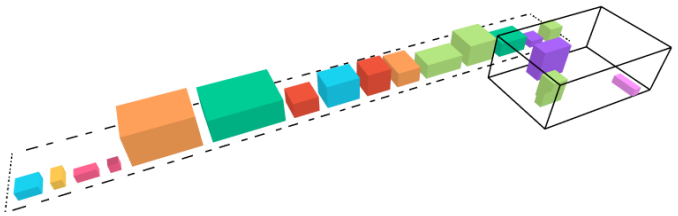
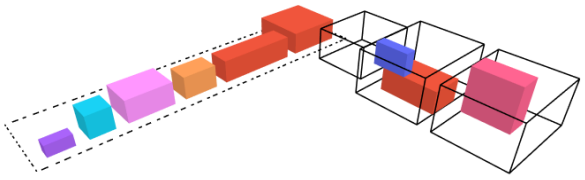
	Objective History	
2	Runtime	104.318 s
	Total Iterations	24
	Final Objective	1320
	Visualization	
	Objective History	
3	Runtime	3.227 s
	Total Iterations	17

	Final Objective	485.00
	Visualization	
	Objective History	

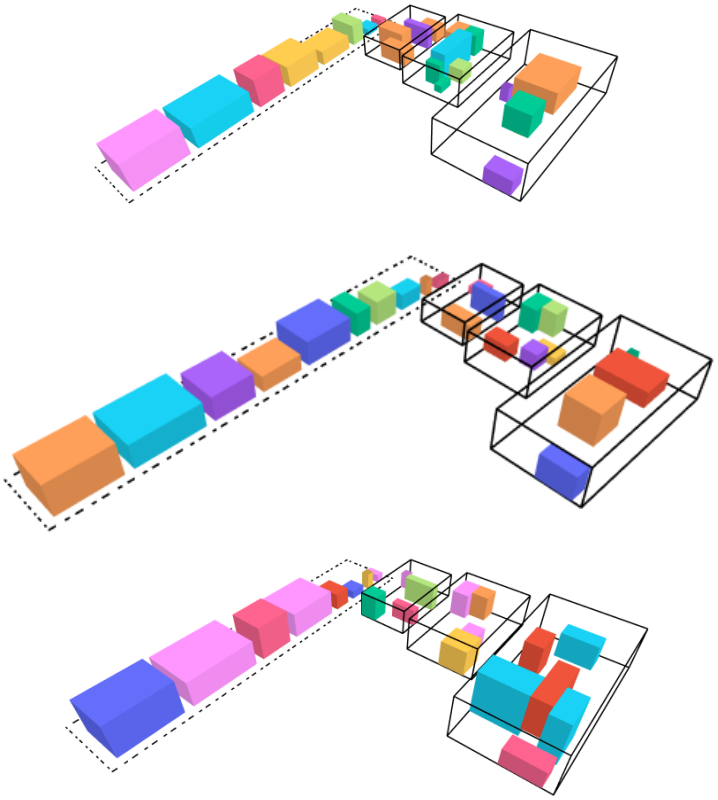
Algoritma *sideways move hill climbing* memungkinkan gerakan menyamping. Akibatnya, algoritma ini memiliki durasi pencarian hampir dua kali lebih lama dari *steepest ascent hill climbing*. Walau demikian, bila dilihat pada kurva objektif, terlihat bahwa pada dataset ini algoritma sudah mencapai lokal optima dan tidak terdapat *state* di sekitarnya dengan objektif yang lebih tinggi, sehingga durasi yang lama ini disebabkan oleh gerakan menyamping yang redundan. Cara menangani masalah ini sebenarnya dapat dilakukan dengan menurunkan batas maksimum gerakan menyamping, tetapi sebaliknya keputusan ini dapat meningkatkan kemungkinan algoritma tidak mencapai *state* selanjutnya yang memiliki objektif lebih tinggi. Sama seperti algoritma *steepest ascent hill climbing*, algoritma ini juga bersifat repetitif sehingga bila dilakukan repetisi tiga kali sekalipun hasilnya selalu sama. Hal ini juga disebabkan karena *sideways move hill climbing* menggunakan *highest valued successor* sebagai penerus *current state*.

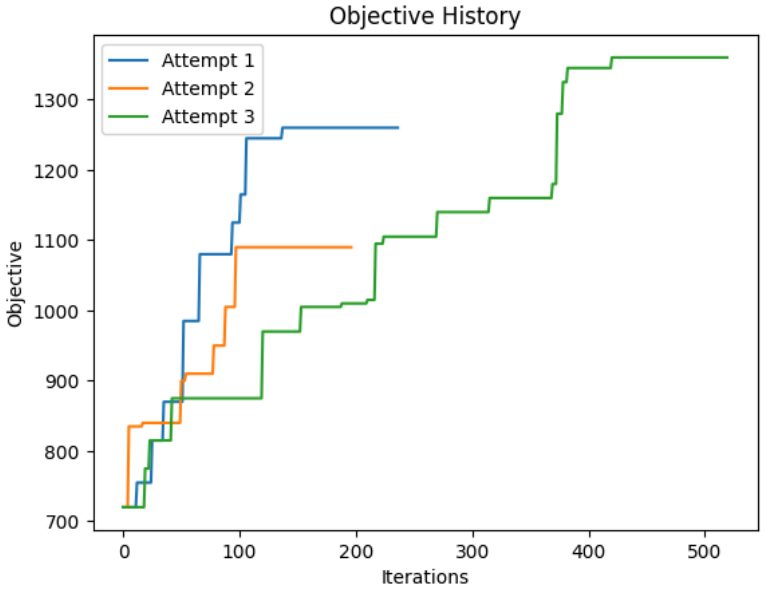
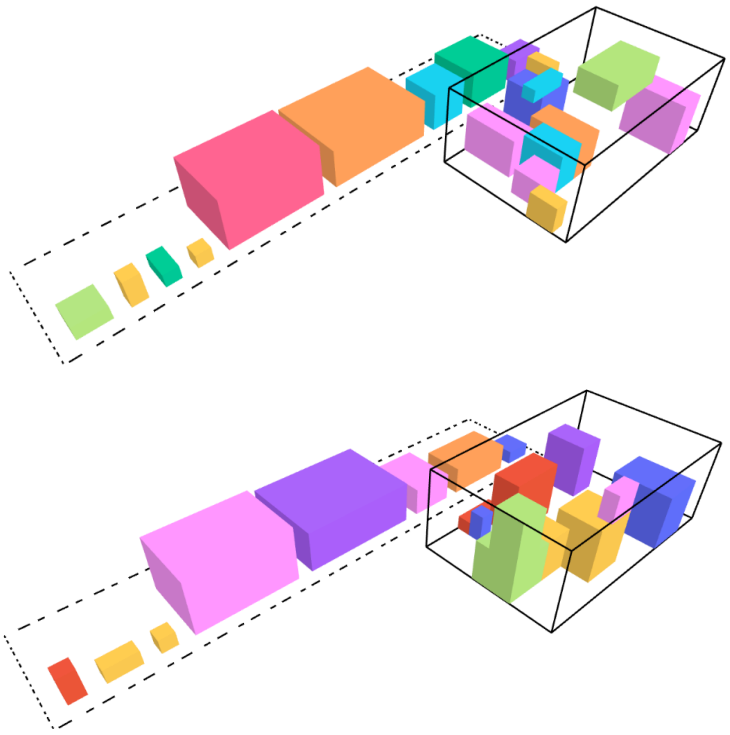
3.1.3. Stochastic

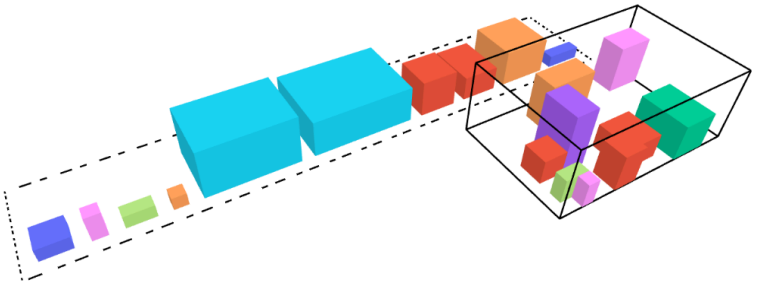
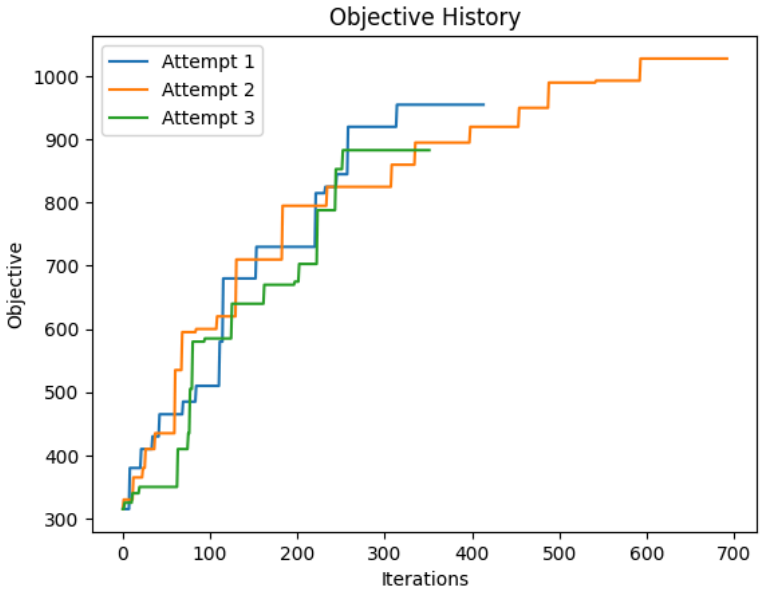
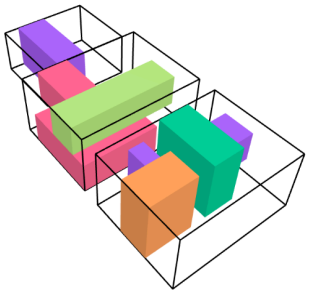
Algoritma *stochastic hill climbing* dilakukan pada tiga dataset beragam berikut (yang menguji kombinasi banyak truk dan paket yang berbeda).

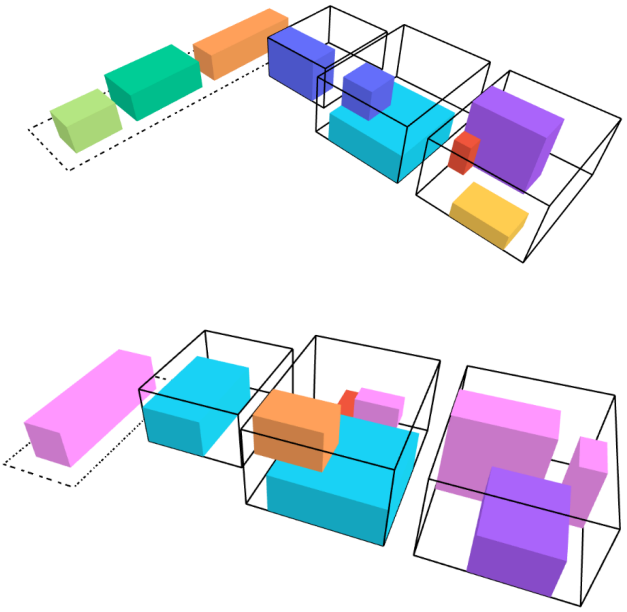
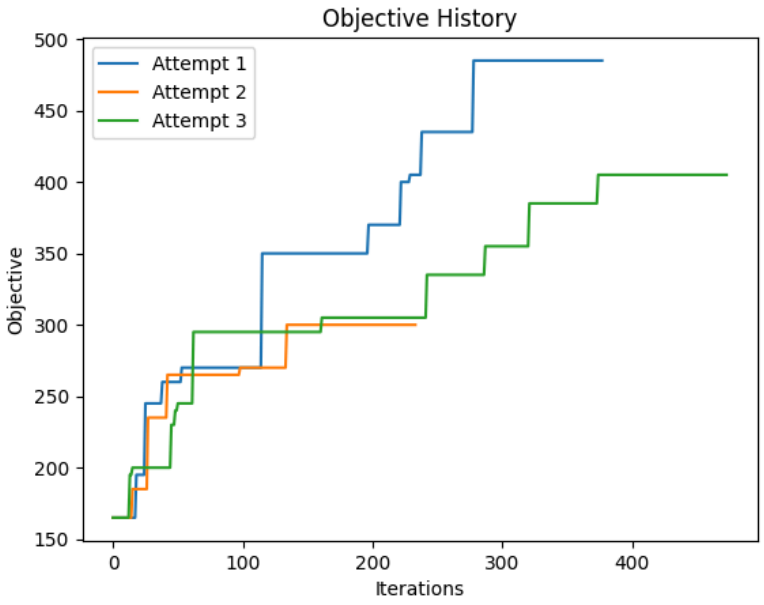
Dataset	Information	Value
1	Truck Count	3
	Package Count	23
	Initial Objective	720
	Visualization	
2	Truck Count	1
	Package Count	19
	Initial Objective	315
	Visualization	
3	Truck Count	3
	Package Count	9
	Initial Objective	165
	Visualization	

Algoritma di-*run* tiga kali pada setiap *dataset* untuk melihat efek faktor *randomness*. *Stochastic hill climbing* menggunakan iterasi *random successor* maksimum sebesar 100. Hasilnya dirangkum pada tabel berikut.

Dataset	**Information**	**Value**
1	Runtime	0.013 s, 0.010 s, 0.025 s
	Total Iterations	237, 197, 520
	Final Objective	1260, 1090, 1360
	Visualization	

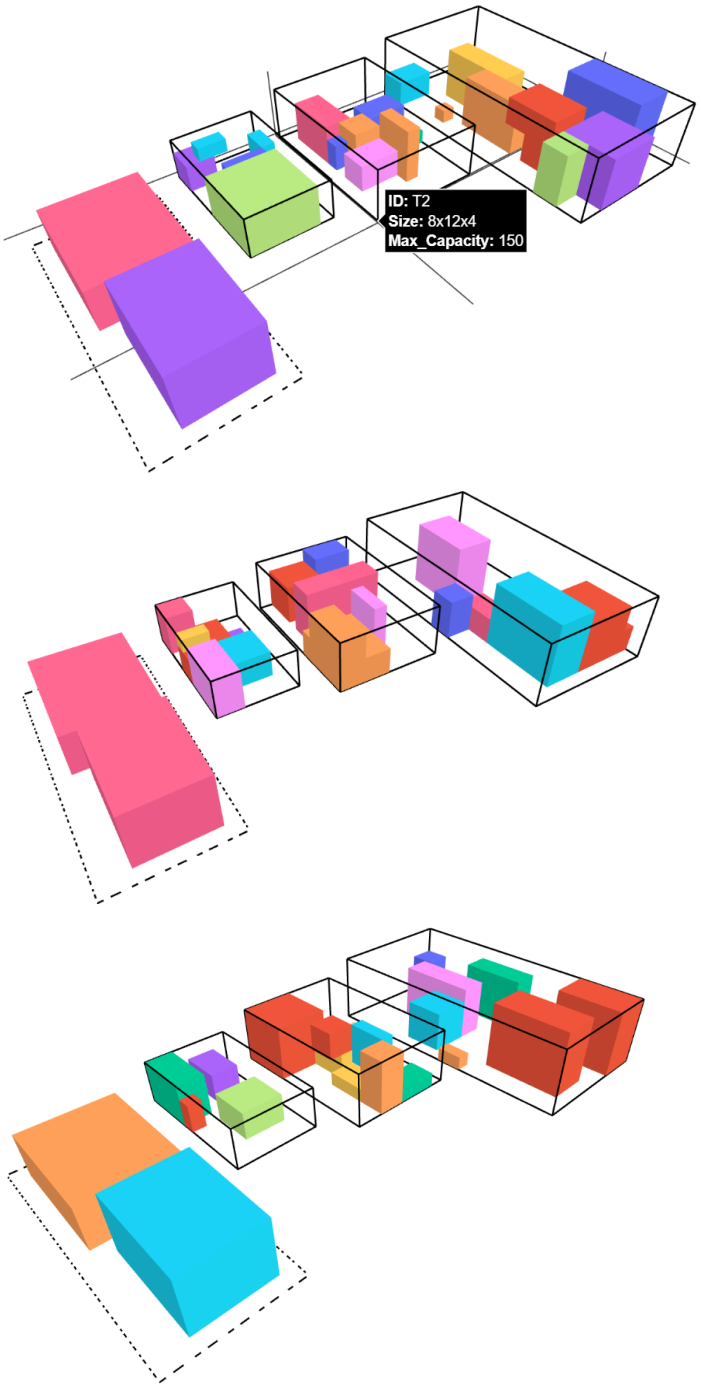
	Objective History	
2	Runtime	0.017s, 0.029s, 0.013s
	Total Iterations	414, 693, 352
	Final Objective	955.00, 1028.00, 883.00
	Visualization	

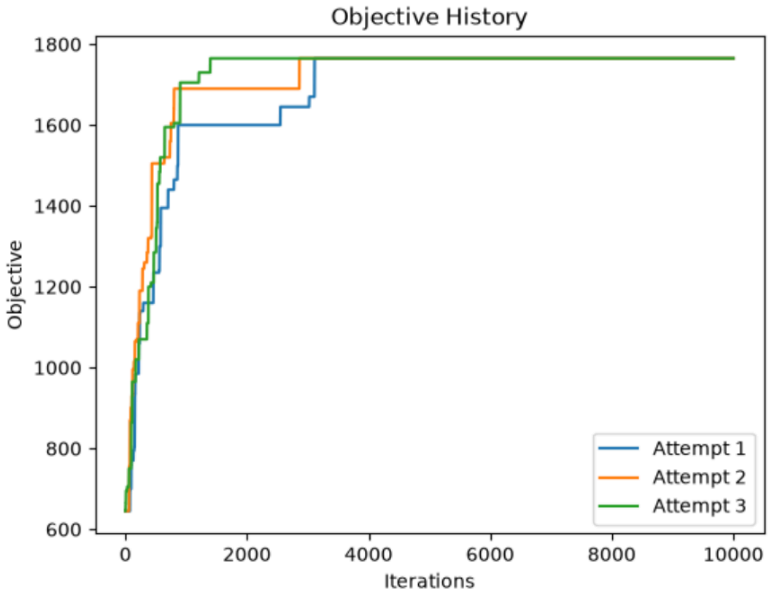
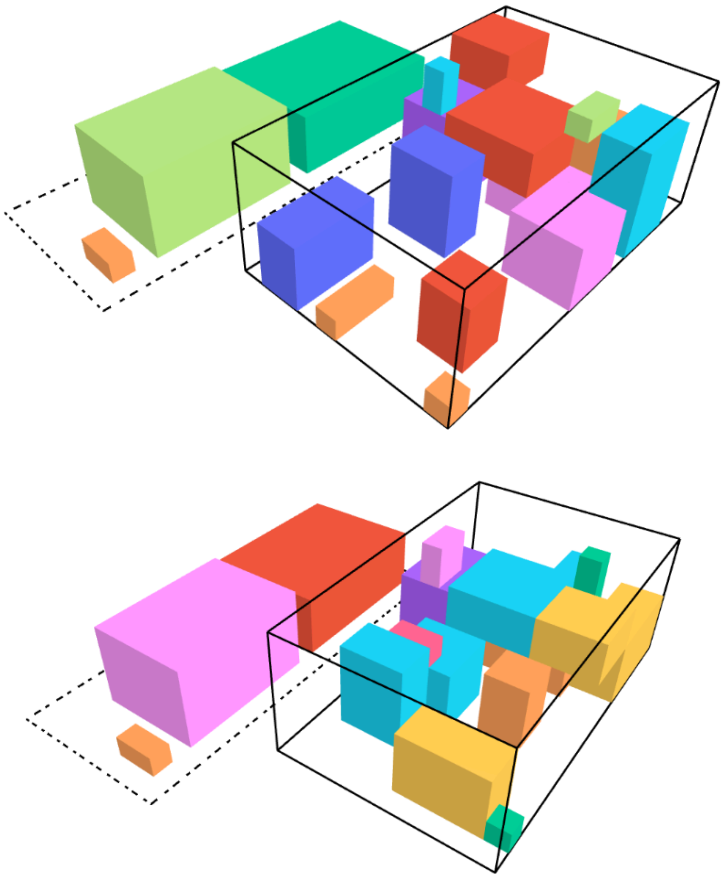
		
	Objective History	
3	Runtime	0.008s, 0.004s, 0.010s
	Total Iterations	378, 234, 474
	Final Objective	485.00, 300.00, 405.00
	Visualization	

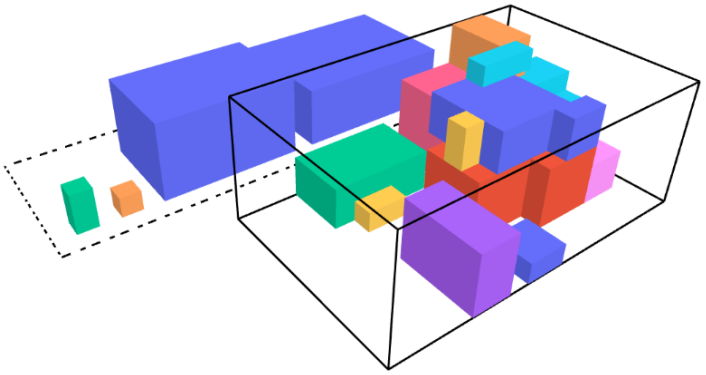
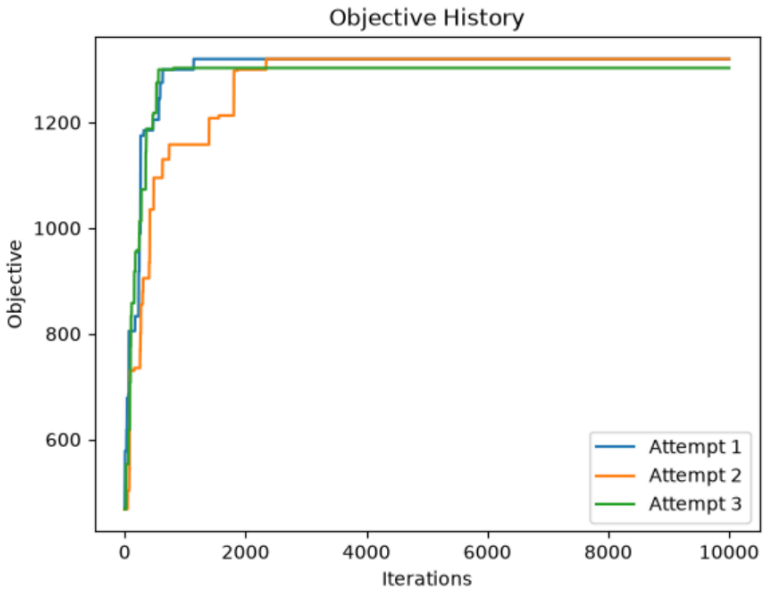
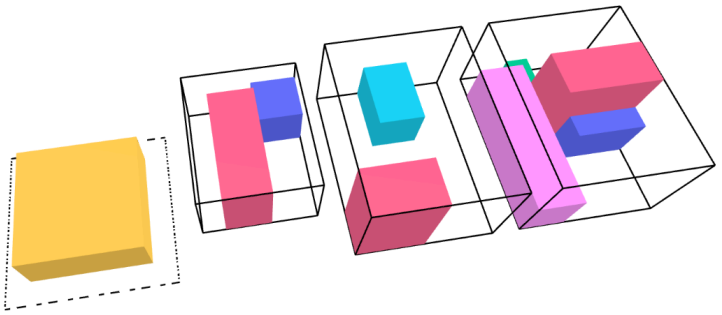
		
	Objective History	

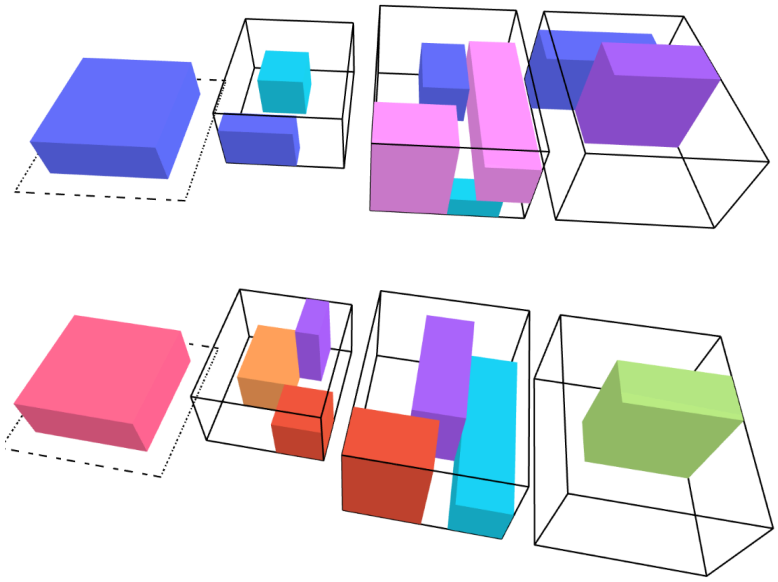
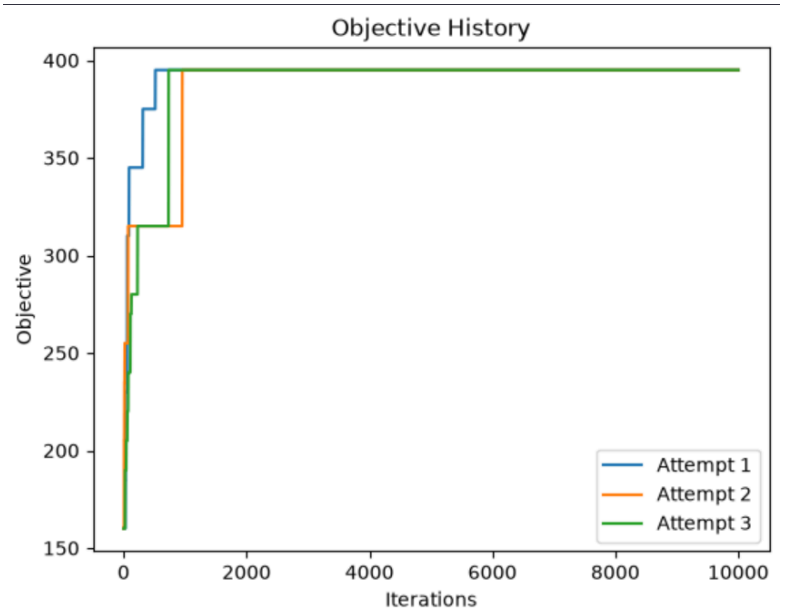
Algoritma *stochastic hill climbing* berbeda dengan kedua algoritma *hill climbing* sebelumnya. Algoritma ini memiliki durasi pencarian yang jauh lebih cepat, tapi sangat tidak konsisten dalam hasil pencariannya. Algoritma ini juga lebih mudah untuk *stuck* di local optima dibanding kedua algoritma *hill climbing* sebelumnya, sebab algoritma ini membangkitkan *random neighbor*. Agar dapat memaksimalkan algoritma ini untuk mendekati global optima, diperlukan jumlah iterasi dan jumlah pembangkitan *random neighbor* yang lebih banyak. Hasil *stochastic hill climbing* dengan jumlah iterasi

pembangkitan *random successor* yang lebih banyak (10000 iterasi) dapat dilihat pada tabel di bawah ini.

Dataset	Information	Value
1	Runtime	0.859s, 0.743s, 0.810s
	Total Iterations	10,000, 10,000, 10,000
	Final Objective	1765.00, 1765.00, 1765.00
	Visualization	

	Objective History	
2	Runtime	1.038s, 0.993s, 0.971s
	Total Iterations	10,000, 10,000, 10,000
	Final Objective	1320.00, 1320.00, 1303.00
	Visualization	

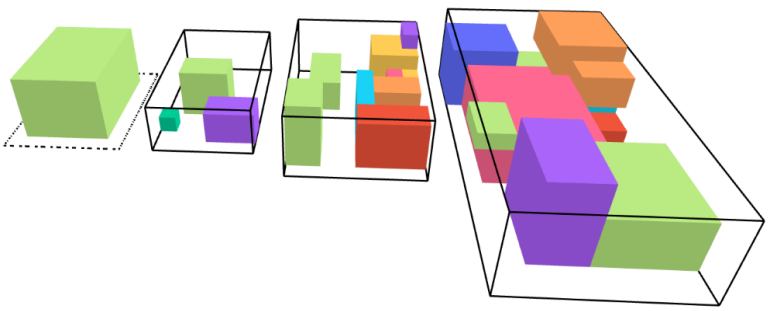
		
	Objective History	
3	Runtime	0.330s, 0.284s, 0.295s
	Total Iterations	10,000, 10,000, 10,000
	Final Objective	395.00, 395.00, 395.00
	Visualization	

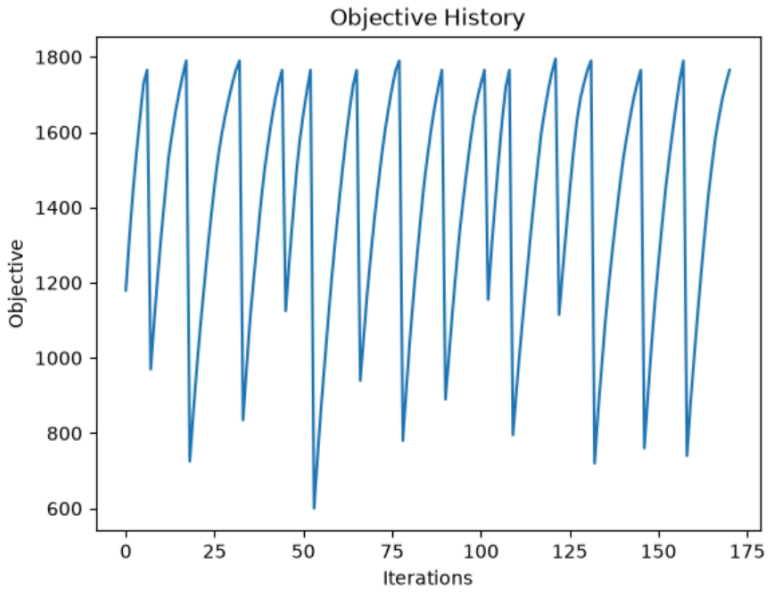
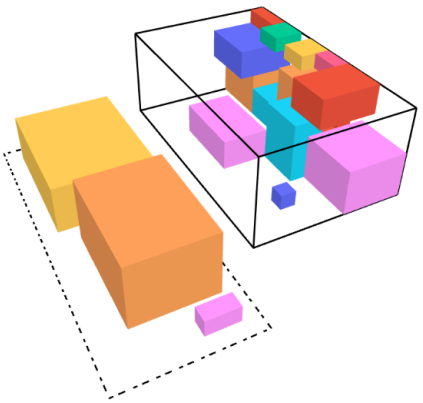
		
	Objective History	

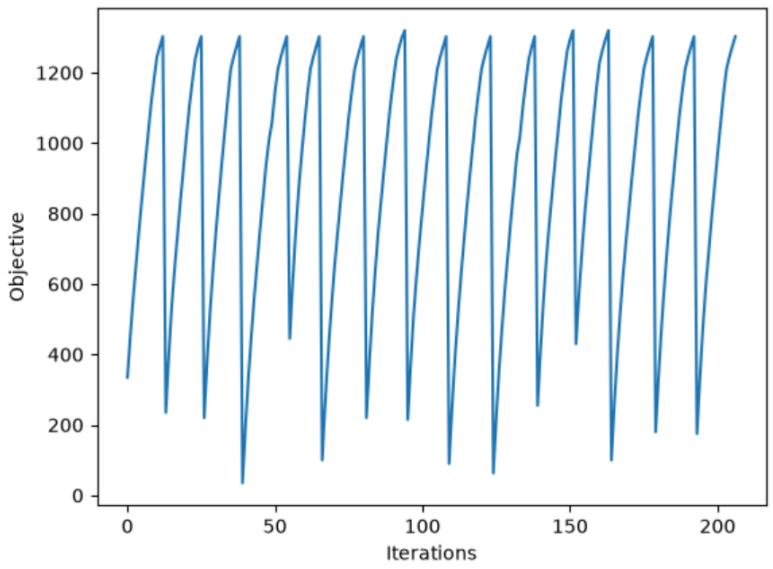
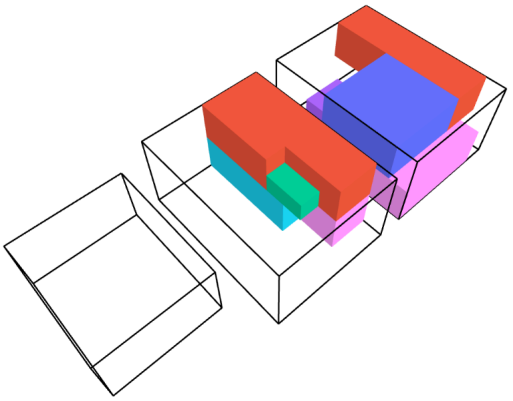
Setelah meningkatkan jumlah iterasi pembangkitan *random successor*, terlihat bahwa algoritma mampu mencapai lokal optima yang lebih tinggi mendekati global optima. Durasi pencarian dari algoritma ini untuk mendekati global optima pun jauh lebih cepat dibanding kedua algoritma *hill climbing* sebelumnya, sebab per iterasinya menggunakan waktu yang sangat singkat karena pembangkitan *random successor* daripada *all successor*. Walau demikian, sebagaimana terlihat pada dataset 2 di tabel di atas, algoritma ini tidak sepenuhnya mampu menghindari *stuck* pada local optima. Ketika *path* yang dipilih salah akibat pembangkitan *random successor* yang salah, algoritma ini dapat terjebak pada local optima hingga akhir.

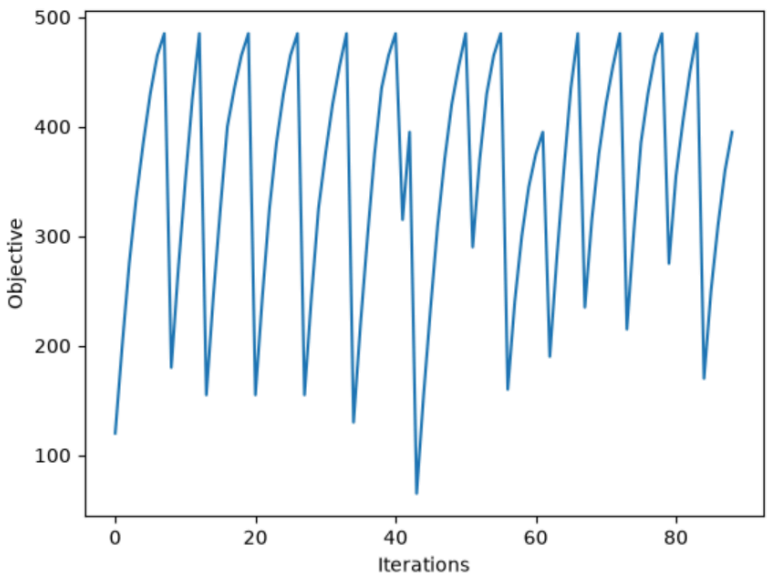
3.1.4. Random Restart

Eksperimen algoritma *random restart hill climbing* dilakukan pada tiga dataset beragam yang sama dengan di atas (yang menguji kombinasi banyak truk dan paket yang berbeda). Detail *initial state* tidak dijabarkan di sini karena hanya suatu *placeholder* bagi algoritma ini. Keunggulan *random restart hill climbing* adalah pembuatan *state* baru yang acak setiap kalinya (yang tidak bisa ditentukan oleh pengguna). Algoritma di-*run* satu kali pada setiap *dataset* karena *random restart* membutuhkan sumber daya komputasi yang sangat mahal, yakni berupa *steepest ascent* yang dilakukan berulang kali. Jumlah *restart* yang dilakukan pada pengujian algoritma ini adalah sebanyak 15 kali *restart*. Hasilnya dirangkum pada tabel berikut.

Dataset	Information	Value
1	Runtime	1124.558s
	Total Iterations per Restart	[7, 11, 15, 12, 8, 13, 12, 12, 12, 7, 13, 10, 14, 12, 13]
	Final Objective	1795.00
	Visualization	

	Objective History	
2	Runtime	732.076s
	Total Iterations per Restart	[13, 13, 13, 16, 11, 15, 14, 14, 15, 15, 13, 12, 15, 14, 14]
	Final Objective	1320.00
	Visualization	

	Objective History	
3	Runtime	15.616s
	Total Iterations per Restart	[8, 5, 7, 7, 7, 7, 2, 8, 5, 6, 5, 6, 6, 5, 5]
	Final Objective	485.00
	Visualization	

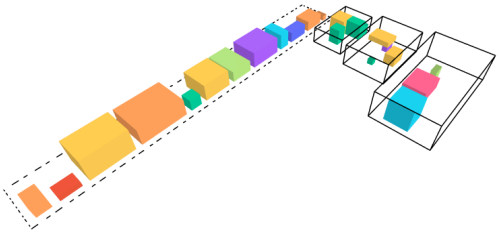
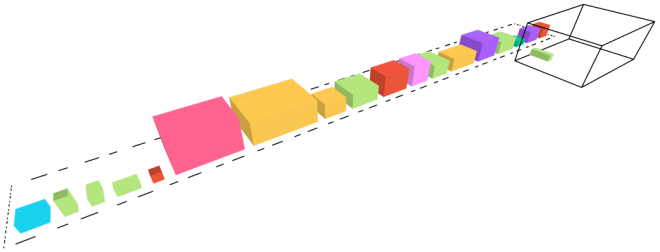
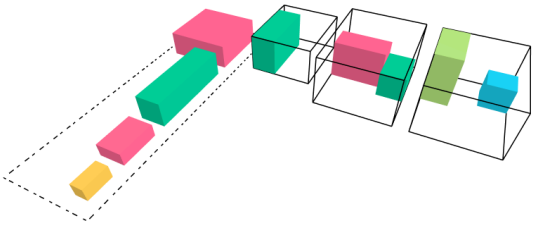
	Objective History	

Algoritma *random restart hill climbing* merupakan algoritma terbaik untuk keluar dari local optima mendekati global optima. Hal ini disebabkan algoritma ini menghasilkan *initial state* baru pada setiap *restart*-nya, sehingga dapat mulai dari *state* yang sepenuhnya berbeda dari *state* awal. Akan tetapi, dapat dilihat bahwa secara komputasi algoritma ini sangat mahal, serta memakan waktu pencarian yang sangat lama dibanding algoritma *hill climbing* lainnya. Dapat dilihat bahwa sekali *restart*, algoritma ini dapat melakukan hingga belasan iterasi *steepest ascent* per *restart*-nya. Oleh sebab itu, algoritma ini memerlukan durasi pencarian selama algoritma *steepest ascent* dikali jumlah *restart* yang ditentukan.

3.2. Simulated Annealing

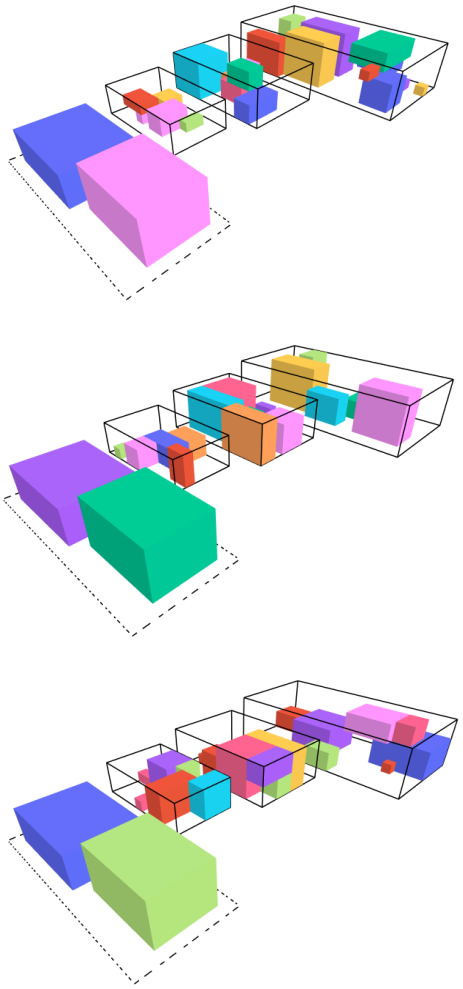
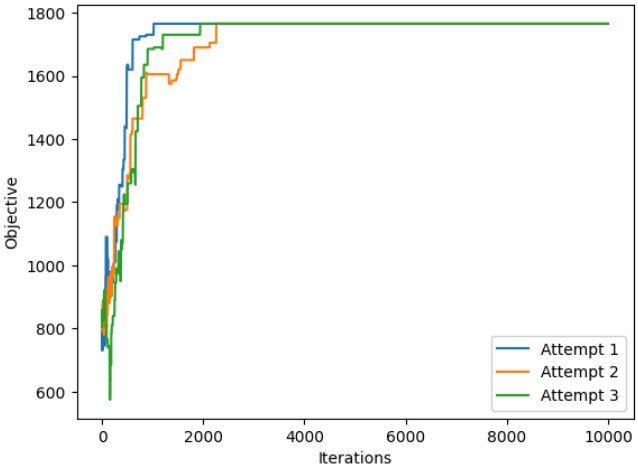
Eksperimen algoritma *simulated annealing* dilakukan pada tiga dataset beragam berikut (yang menguji kombinasi banyak truk dan paket yang berbeda).

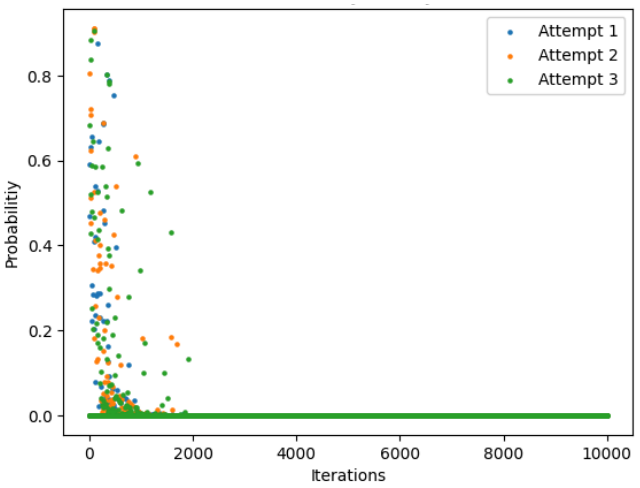
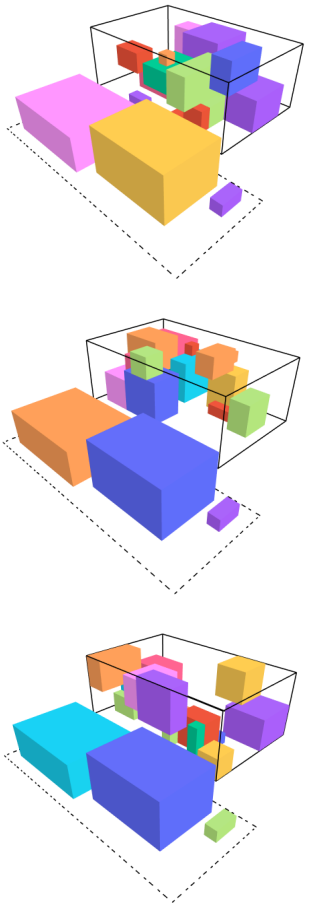
Dataset	Information	Value
1	Truck Count	3
	Package Count	23
	Initial Objective	805.00

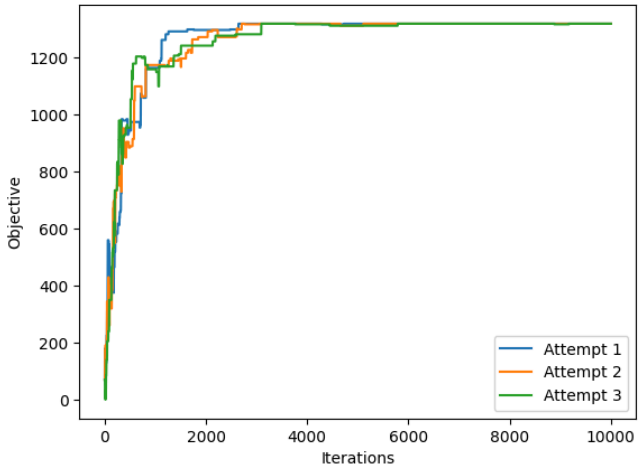
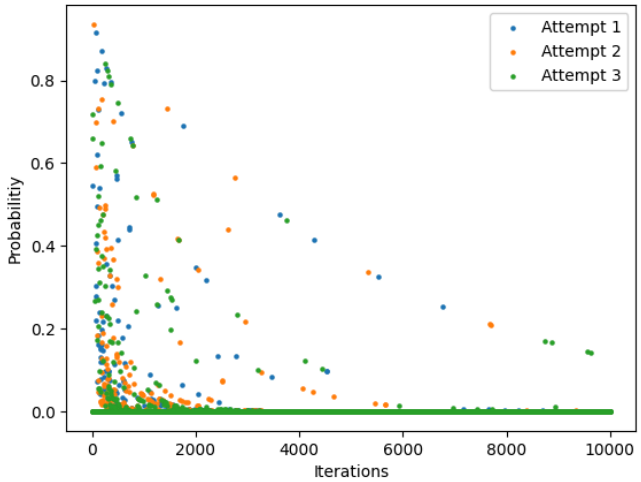
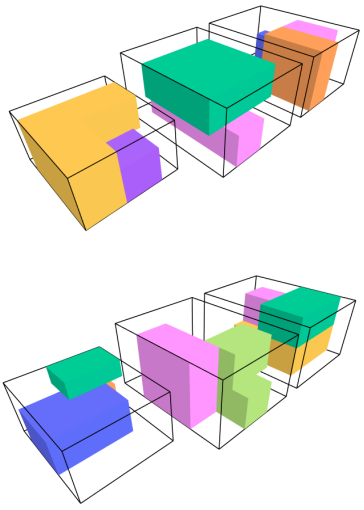
	Visualization	
2	Truck Count	1
	Package Count	19
	Initial Objective	70.00
	Visualization	
3	Truck Count	3
	Package Count	9
	Initial Objective	265.00
	Visualization	

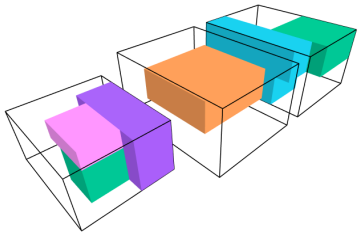
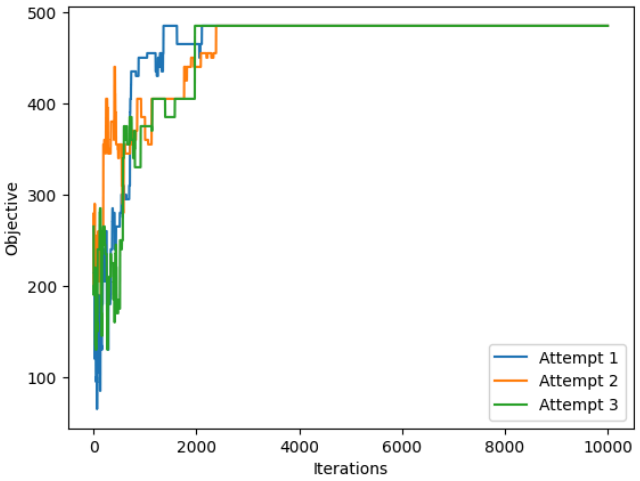
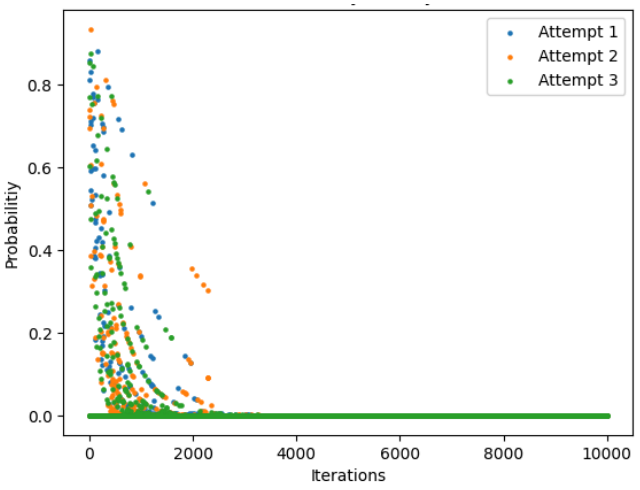
Algoritma di-*run* tiga kali pada setiap *dataset* untuk menguji pengaruh *randomness* pada algoritma. Hasilnya dirangkum pada tabel berikut.

Dataset	**Information**	**Value**
1	Runtime	2.385s, 2.323s, 2.219s
	Total Iterations	10,000, 10,000, 10,000
	Final Objective	1765.00, 1765.00, 1765.00

	Local Optima	14, 14, 14
	Visualization	
	Objective History	

	Probability History	
2	Runtime	2.674s, 2.648s, 2.739s
	Total Iterations	10,000, 10,000, 10,000
	Final Objective	1320.00, 1320.00, 1320.00
	Local Optima	16, 22, 15
	Visualization	

	Objective History	
	Probability History	
3	Runtime	0.692s, 0.687s, 0.685s
	Total Iterations	10,000, 10,000, 10,000
	Final Objective	485.00, 485.00, 485.00
	Local Optima	25, 22, 19
	Visualization	

		
	Objective History	
	Probability History	

Berdasarkan hasil di atas, terlihat bahwa algoritma *simulated annealing* sangat efisien, dengan durasi *runtime* yang sangat cepat bahkan untuk *dataset* pertama yang berat. Walaupun membutuhkan sangat banyak iterasi untuk konvergen (dalam orde ribuan, dibandingkan *hill climbing* yang hanya memerlukan beberapa puluh iterasi), kecepatannya membuat algoritma ini sangat menarik. Selebihnya, di setiap *dataset*, pencobaan ulang menghasilkan solusi akhir yang serupa, walaupun terlihat dari *objective history* perbedaan pendekatannya.

Perlu dicatat juga bahwa efek temperatur (yang diatur oleh *schedule*) sangat terlihat dari sejarah nilai objektif dan peluang. Di iterasi-iterasi awal (0 hingga 2000), temperatur yang masih tinggi memungkinkan algoritma mengeksplorasi *state-state* yang baru dalam

upaya mencari solusi optimal. Ini mengapa kita melihat banyak *local optimum* di iterasi-iterasi awal tersebut. Tetapi, dibandingkan jumlah iterasi total hingga mencapai solusinya (dalam ribuan), banyak iterasi *local optimum* (dalam puluhan) lumayan kecil, sehingga algoritma *simulated annealing* terlihat melakukan *climbing* dengan efektif bahkan saat temperatur besar.

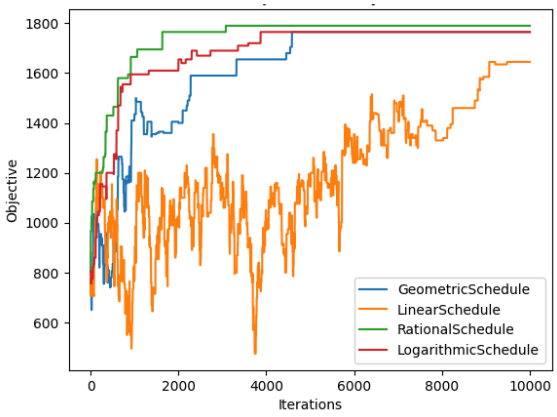
Selebihnya, berikutnya dilakukan juga perbandingan keempat fungsi *schedule* yang mungkin dengan parameter sebagai berikut:

- `GeometricSchedule` dengan $T_0 = 100,\ \alpha = 0.99,$
- `LinearSchedule` dengan $T_0 = 100,\ \beta = 0.01,$
- `RationalSchedule` dengan $T_0 = 100,\ \beta = 0.001,$
- `LogarithmicSchedule` dengan $T_0 = 100,\ c = ln(2),\ t_0 = 2.$

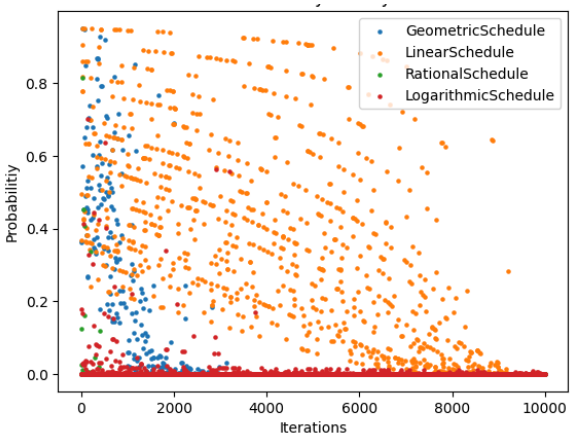
Sebagai kontrol, tetap digunakan fungsi objektif `MaxValueObjective`. Pengujian dilakukan pada *dataset* pertama dengan 3 truk dan 23 paket (lihat tabel di atas). Hasil dari eksperimen ini sebagai berikut.

Schedule	Runtime	Total Iterations	Final Objective	Local Optimum
Geometric	2.131s	10,000	1765.00	29
Linear	1.577s	10,000	1645.00	184
Rational	2.436s	10,000	1790.00	3
Logarithmic	2.264s	10,000	1765.00	9

Objective History



Probability History

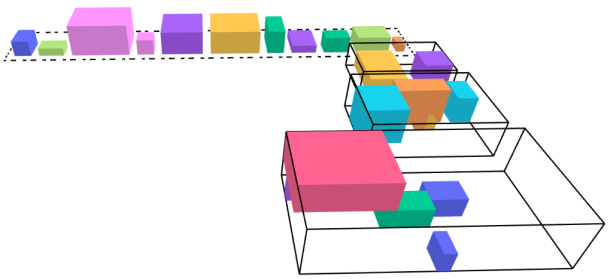
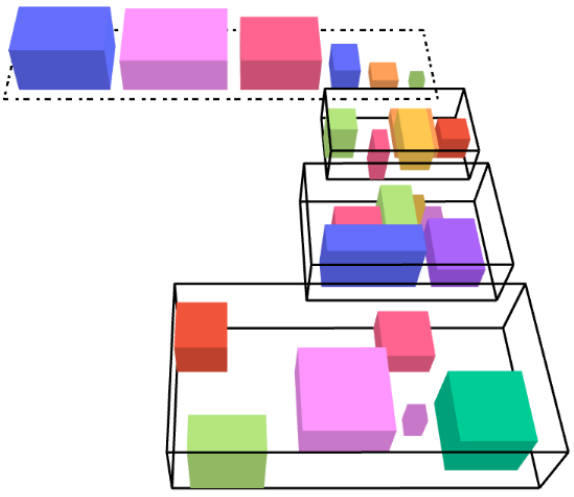


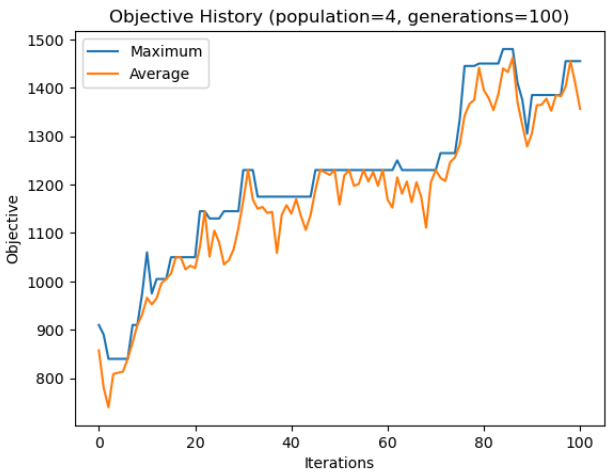
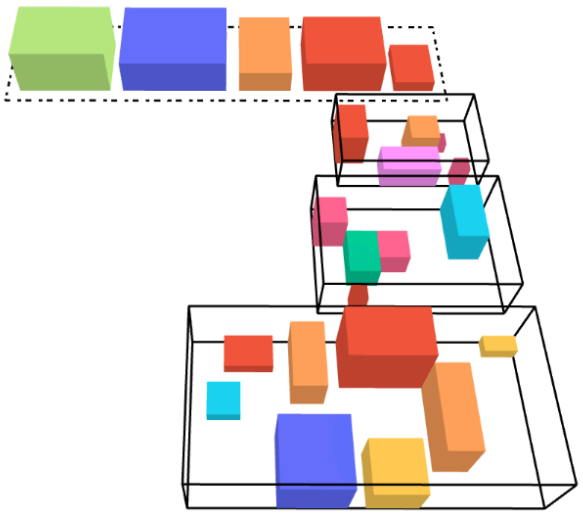
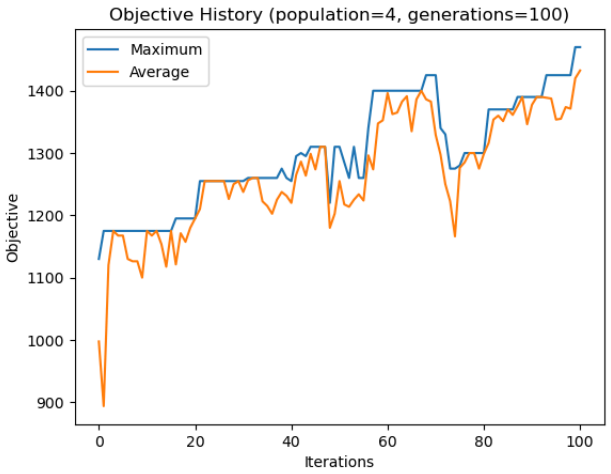
Berdasarkan hasil di atas, terlihat bahwa *schedule* yang berbeda memanipulasikan temperatur dengan berbeda, yang sangat berpengaruh pada algoritma *simulated annealing*. Di

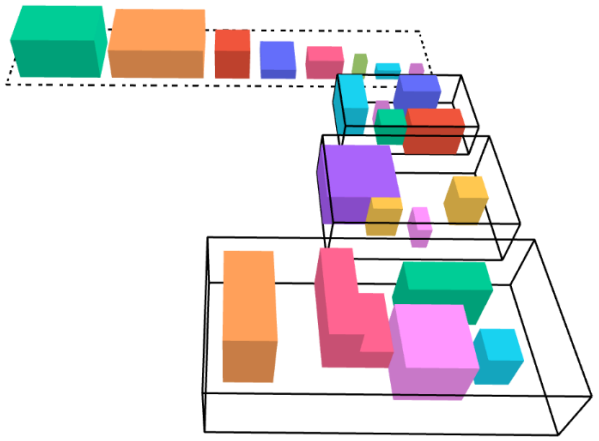
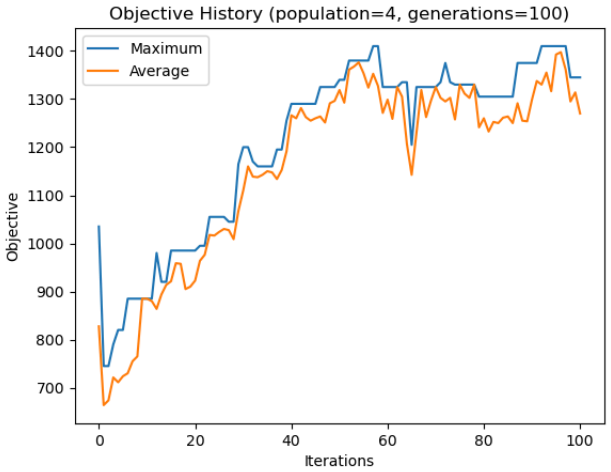
sini, *schedule* rational yang paling unggul, dengan nilai objektif akhir tertinggi dan *stuck* optimum lokal paling jarang. Di sisi lain, *schedule* linear memiliki performa paling buruk, karena mengurangi temperatur secara linear terlalu lambat sehingga memberikan terlalu banyak kesempatan bagi *simulated annealing* untuk melompat-melompat pada *state* yang kurang optimal (sehingga banyak *local optimum*). Ini juga terlihat di sejarah nilai peluang di mana di iterasi yang sangat besar masih tinggi peluang *state transition*-nya. Terakhir, *schedule* logarithmic dan geometri juga memiliki performa yang lumayan bagus, seperti terlihatnya bagaimana *simulated annealing* diberikan kesempatan eksplorasi di iterasi-iterasi awal tetapi akhirnya mengerucut pada solusi setelah banyak iterasi.

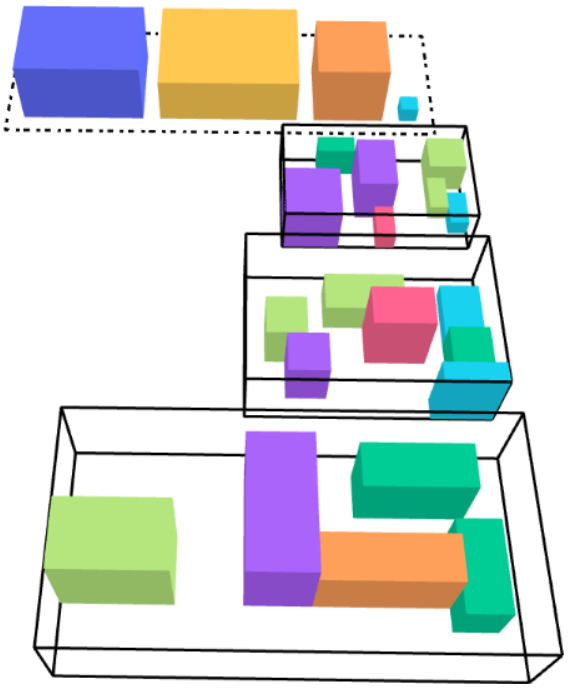
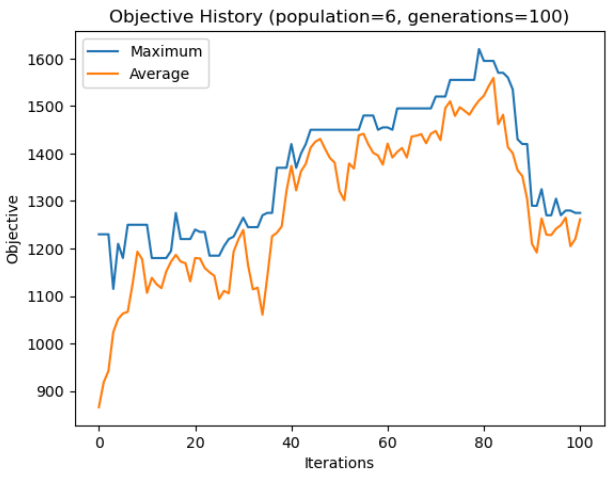
3.3. Genetic Algorithm

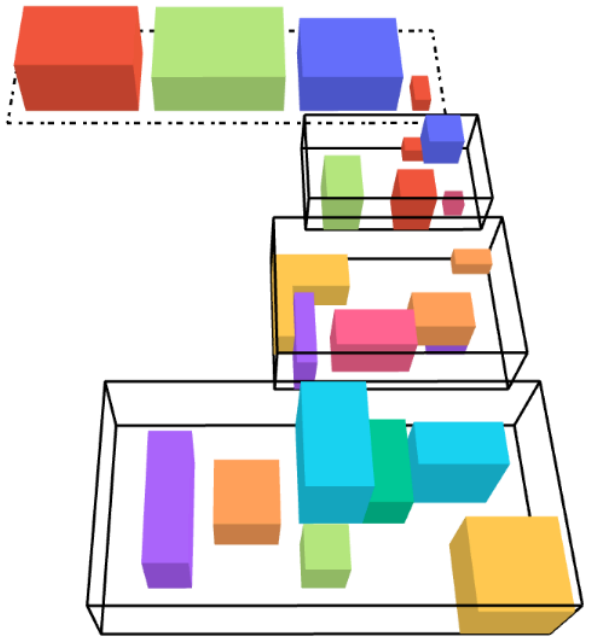
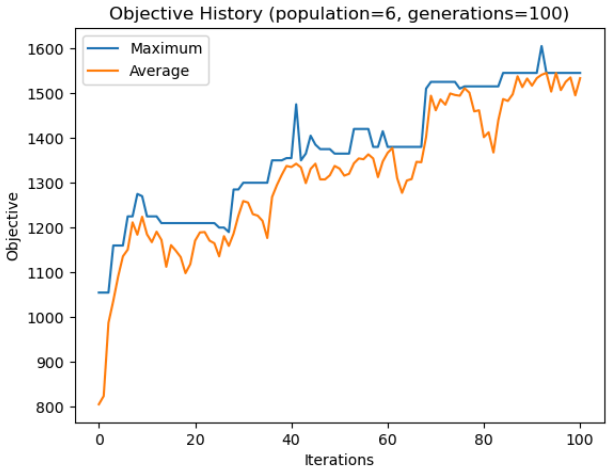
Eksperimen algoritma Genetic Algorithm dilakukan pada sebuah dataset yang sama namun dengan beberapa konfigurasi parameter yang berbeda.

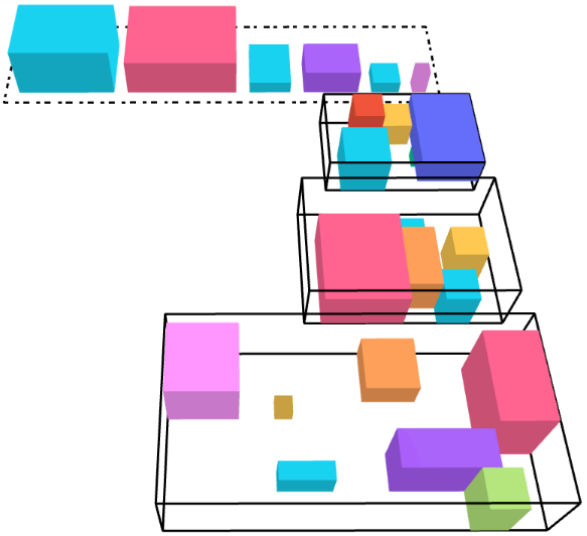
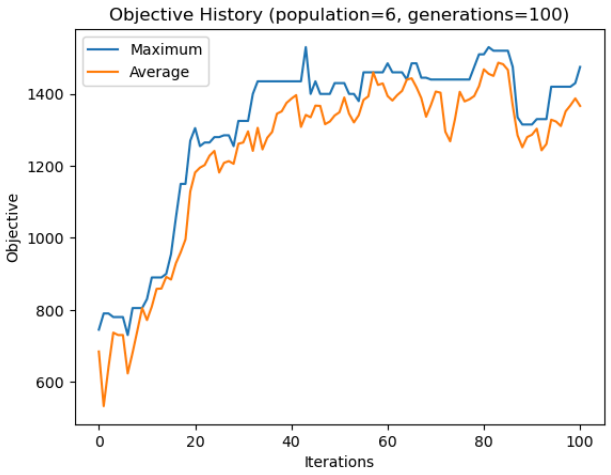
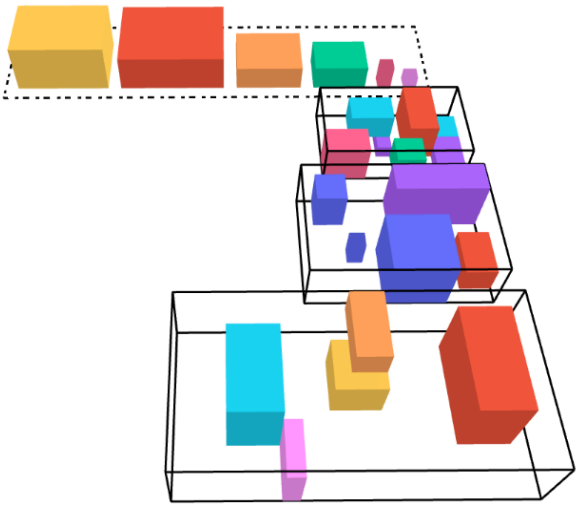
<table>
<tr><td colspan="3">State Awal</td><td>  </td></tr>
<tr><td>Configuration</td><td>Iteration</td><td>Information</td><td>Value</td></tr>
<tr><td rowspan="2">1
Jumlah Populasi: 4
Banyak Iterasi: 100</td><td rowspan="2">1</td><td>State Akhir</td><td>  </td></tr>
<tr><td>Objective Value</td><td>1480</td></tr>
</table>

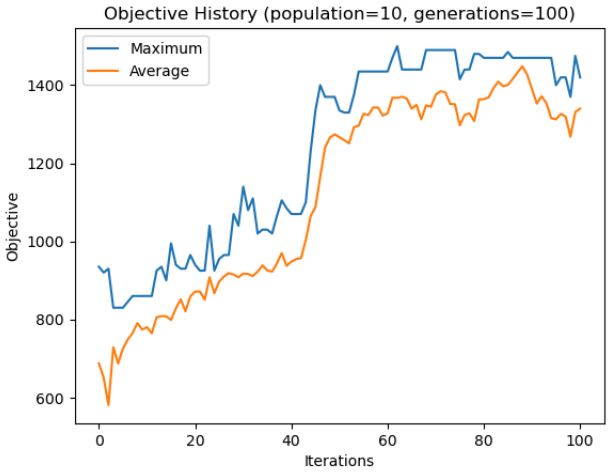
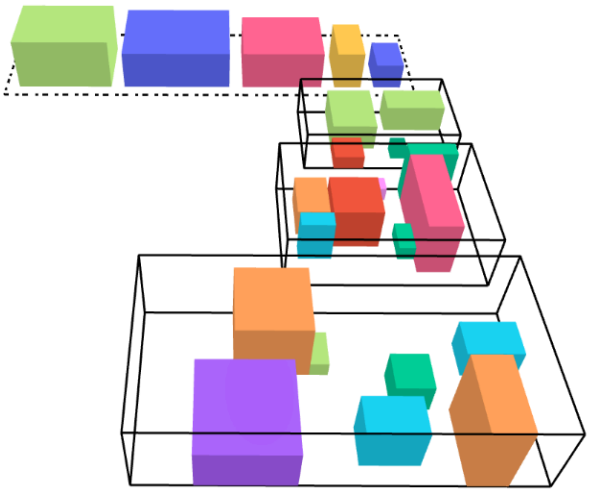
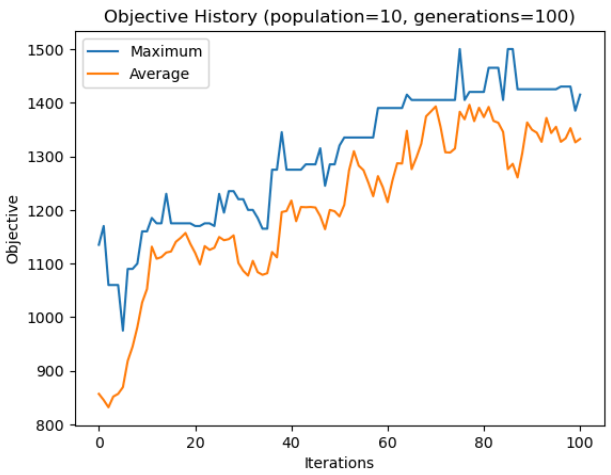
		Plot Objective Function	
		Durasi	0.482s
	2	State Akhir	
		Objective Value	1470
		Plot Objective Function	
		Durasi	0.921s

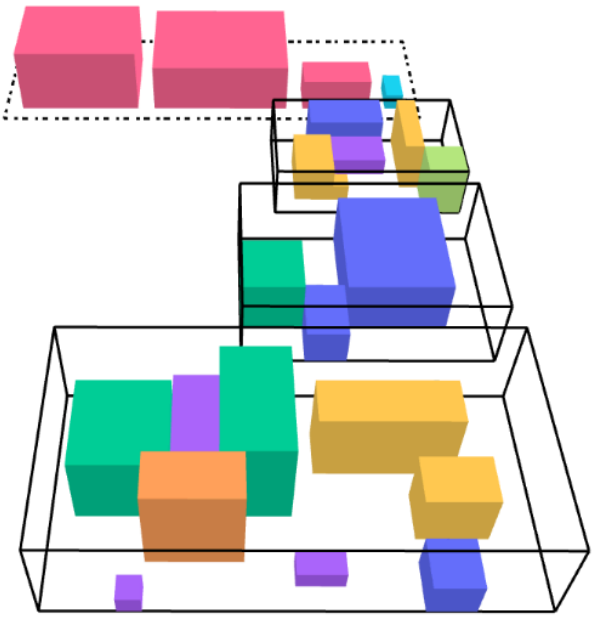
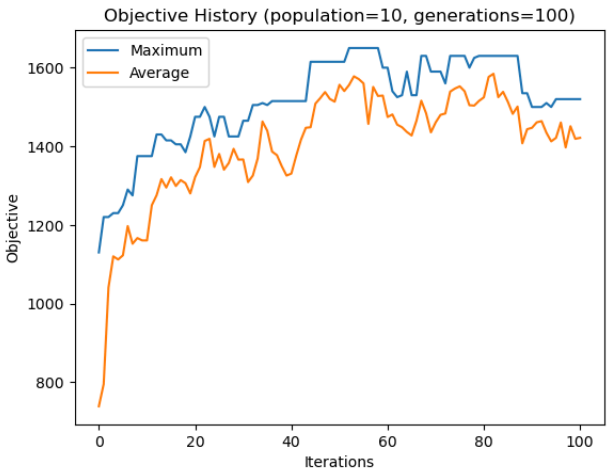
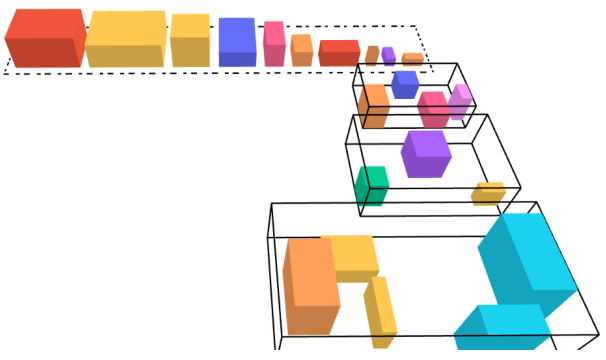
	3	State Akhir	
		Objective Value	1410
		Plot Objective Function	
		Durasi	0.437s

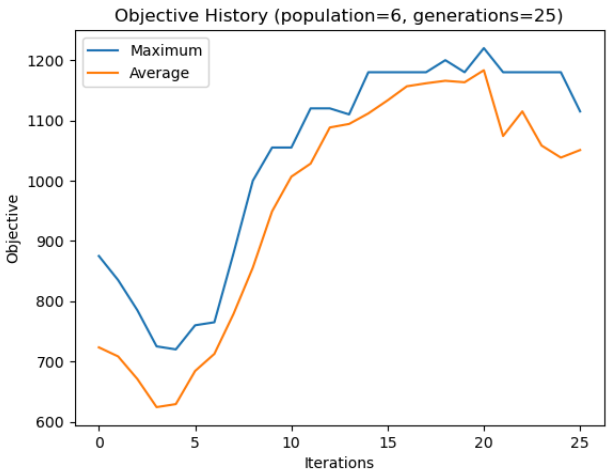
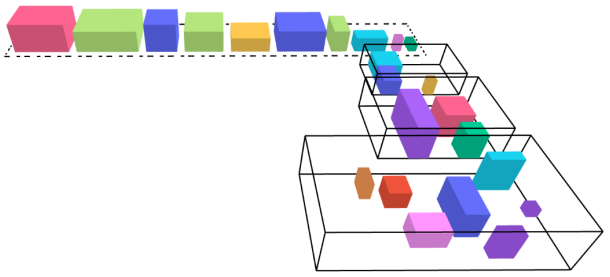
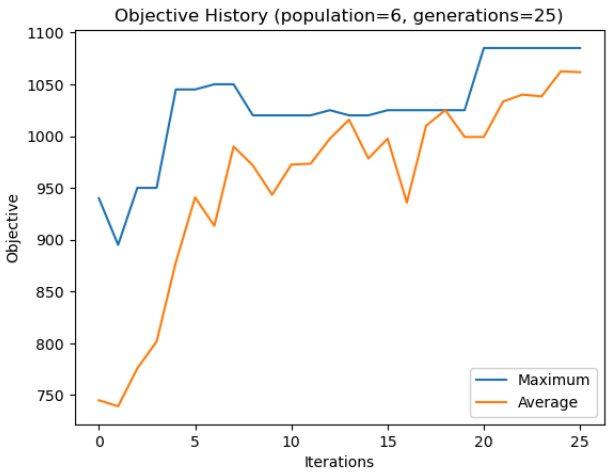
2 (Kontrol) Jumlah Populasi: 6 Banyak Iterasi: 100	1	State Akhir	
		Objective Value	1620
		Plot Objective Function	
		Durasi	0.836s

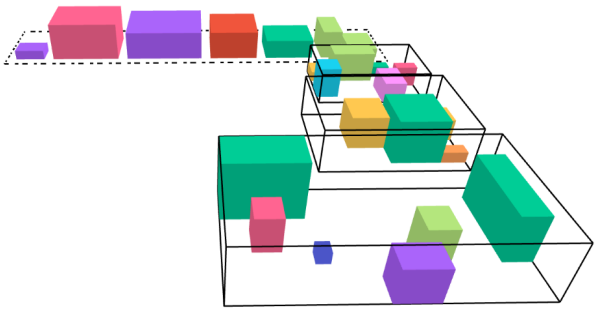
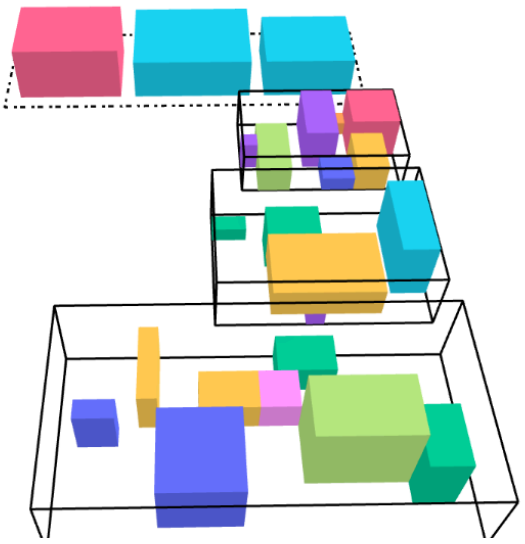
	2	State Akhir	
		Objective Value	1605
		Plot Objective Function	
		Durasi	0.930s

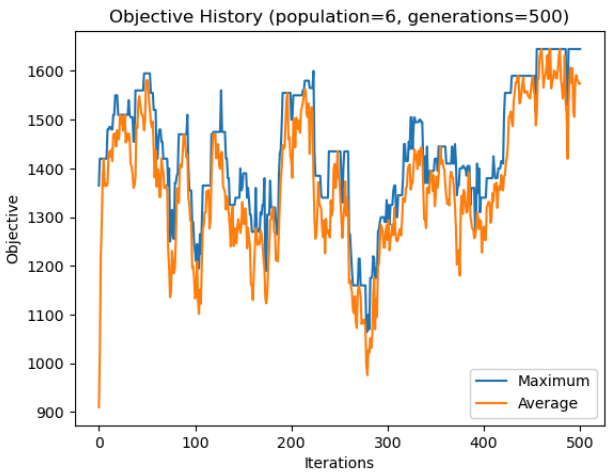
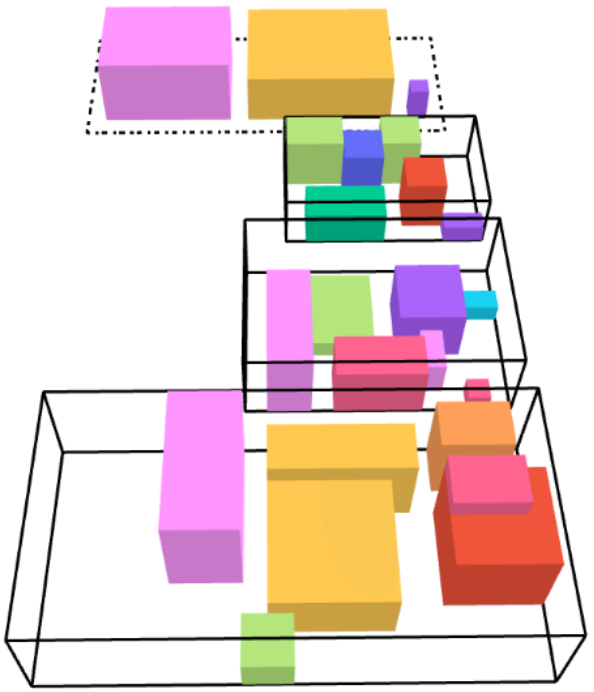
	3	State Akhir	
		Objective Value	1530
		Plot Objective Function	
		Durasi	0.855
3 Jumlah Populasi: 10 Banyak Iterasi: 100	1	State Akhir	

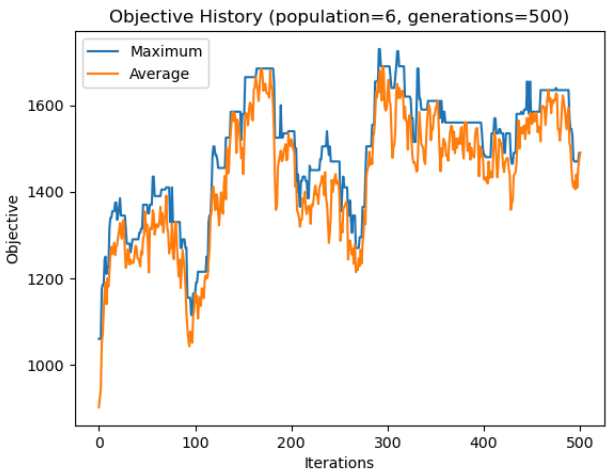
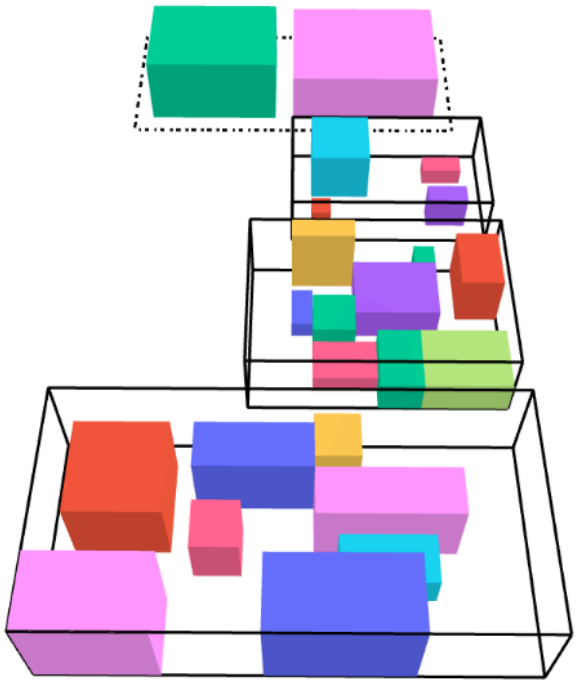
		Objective Value	1500
		Plot Objective Function	
		Durasi	1.170s
	2	State Akhir	
		Objective Value	1500
		Plot Objective Function	

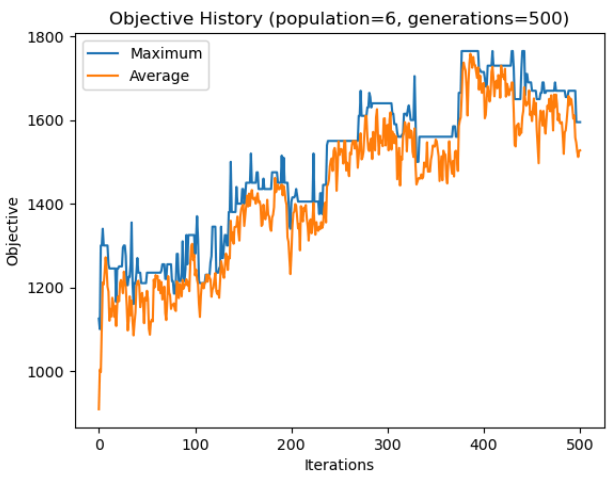
		Durasi	1.194s
	3	State Akhir	
		Objective Value	1650
		Plot Objective Function	
		Durasi	1.845s
4 Jumlah Populasi: 6 Banyak Iterasi: 30	1	State Akhir	

		Objective Value	1220
		Plot Objective Function	
		Durasi	0.261s
	2	State Akhir	
		Objective Value	1085
		Plot Objective Function	
		Durasi	0.257s

	3	State Akhir	
		Objective Value	1270
		Plot Objective Function	
		Durasi	0.147s
5 Jumlah Populasi: 4 Banyak Iterasi: 250	1	State Akhir	
		Objective Value	1645

		Plot Objective Function	
		Durasi	5.022s
	2	State Akhir	
		Objective Value	1730

		Plot Objective Function	
		Durasi	5.534s
	3	State Akhir	
		Objective Value	1765

		Plot Objective Function	
		Durasi	7.061s

Dari iterasi tiap konfigurasi terlihat bahwa final state yang dihasilkan tiap iterasi berbeda meskipun memiliki konfigurasi parameter yang sama. Hal ini mengindikasikan bahwa metode Genetic Algorithm memiliki hasil yang stokastik, dipengaruhi oleh populasi awal, hasil selection, dan hasil mutate.

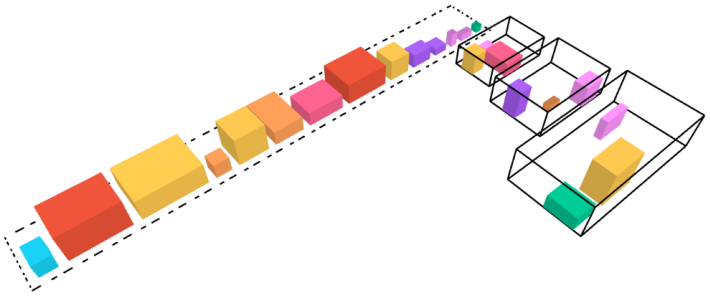
Lalu dari konfigurasi kontrol dengan jumlah populasi 6 dan banyak iterasi 100, dilakukan variasi jumlah populasi. Dari hasil konfigurasi 1, 2, dan 3 terlihat bahwa secara garis besar, jumlah populasi yang sedang di konfigurasi 2 lebih unggul dibandingkan kedua konfigurasi lainnya. Jumlah populasi yang terlalu kecil menyebabkan variasi genetik rendah dan agent lebih mungkin untuk mengalami konvergensi di local maximum. Namun jumlah populasi yang terlalu besar juga menyebabkan ruang pencarian terlalu luas sehingga apabila ada state yang bagus, fungsi objektifnya relatif sangat kecil dibandingkan total fungsi objektif di generasi tersebut sehingga probabilitas state tersebut dipilih juga tetap kecil. Sehingga jumlah populasi yang sebaiknya dipakai adalah yang tidak terlalu tinggi.

Dari kontrol juga dilakukan variasi banyak iterasi. Secara keseluruhan, terlihat bahwa semakin banyak iterasi, maka nilai fungsi objektif akhir semakin tinggi juga. Hal ini terjadi karena dengan iterasi yang banyak, komputer mampu mencari state lebih banyak. Namun banyak iterasi yang tinggi menyebabkan waktu komputasi menjadi lebih lama, meskipun tidak terlalu signifikan.

3.4. Analisis Perbandingan

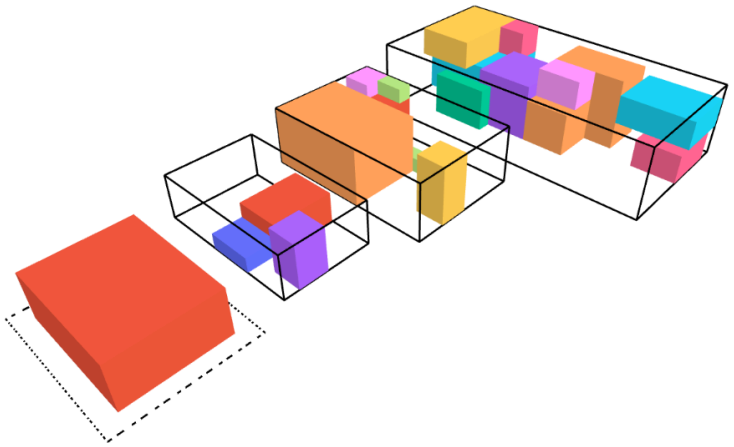
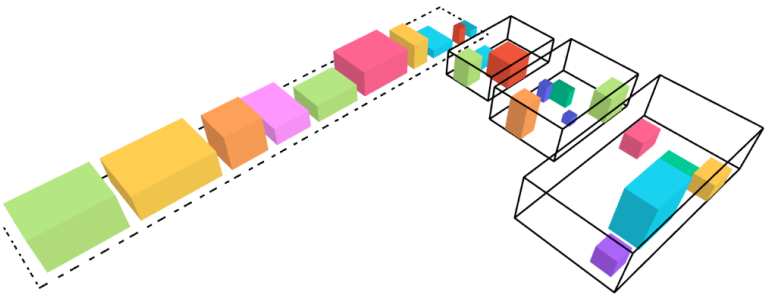
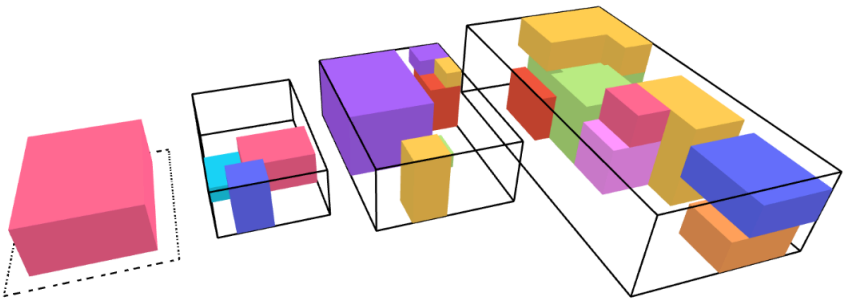
Akhirnya, algoritma *hill climbing*, *simulated annealing*, dan *genetic algorithm* dibandingkan satu dengan yang lain. Iterasi maksimum yang dilakukan adalah sebanyak 200

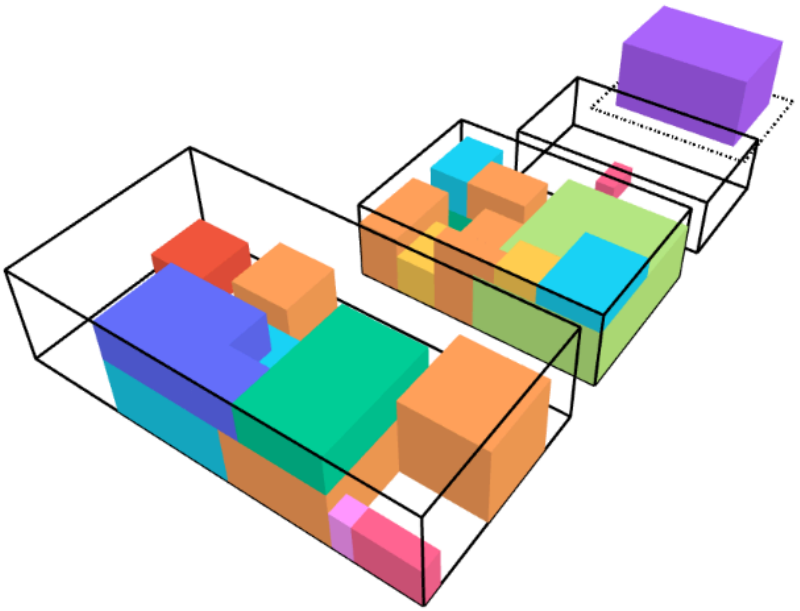
iterasi dan fungsi objektif yang digunakan adalah fungsi `MaxValueObjective`. Pada analisis perbandingan ini, hanya digunakan satu dataset sebagai kontrol, yaitu 3 truk dan 23 paket dengan konfigurasi *initial state* sebagai berikut.

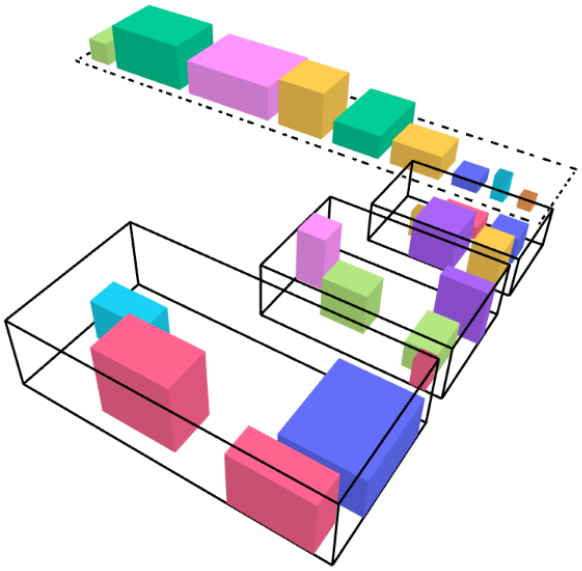
Dataset	**Information**	**Value**
1	Truck Count	3
	Package Count	23
	Initial Objective	790
	Visualization	

Selanjutnya, dilakukan analisis pada dataset ini dengan masing-masing algoritma. Untuk *stochastic hill climbing*, jumlah iterasi *random successor* maksimum adalah 100. Untuk *sideways move hill climbing*, jumlah *sideways move* yang diperbolehkan adalah sebesar 10. Untuk *random restart hill climbing*, dilakukan 15 kali *restart*. Pada *simulated annealing*, digunakan *RationalSchedule* dengan temperatur awal 100 dan nilai beta 0.001. Pada *genetic algorithm*, digunakan ukuran populasi sebesar 5.

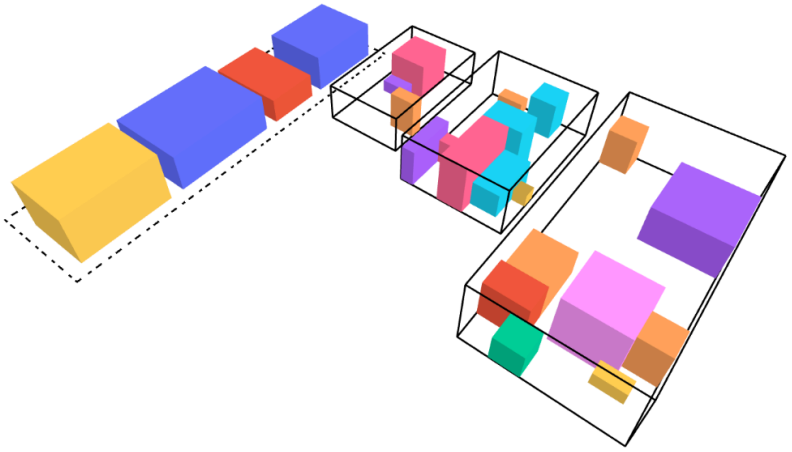
Steepest Ascent Hill Climbing	
Information	**Value**
Runtime	70.800s
Total Iterations	14
Final Objective	1790.00

Visualization	
Stochastic Hill Climbing	
Information	**Value**
Runtime	0.020s
Total Iterations	200
Final Objective	1115.00
Visualization	
Sideways Move Hill Climbing	
Information	**Value**
Runtime	130.893s
Total Iterations	24
Final Objective	1790.00
Visualization	

Random Restart Hill Climbing	
Information	**Value**
Runtime	1333.034s
Total Iterations per Restart	[14, 15, 17, 12, 16, 11, 9, 16, 16, 14, 16, 12, 16, 15, 14]
Final Objective	1795.00
Visualization	
Simulated Annealing	
Information	**Value**
Runtime	0.038s
Iterations	200
Final Objective	1240.00

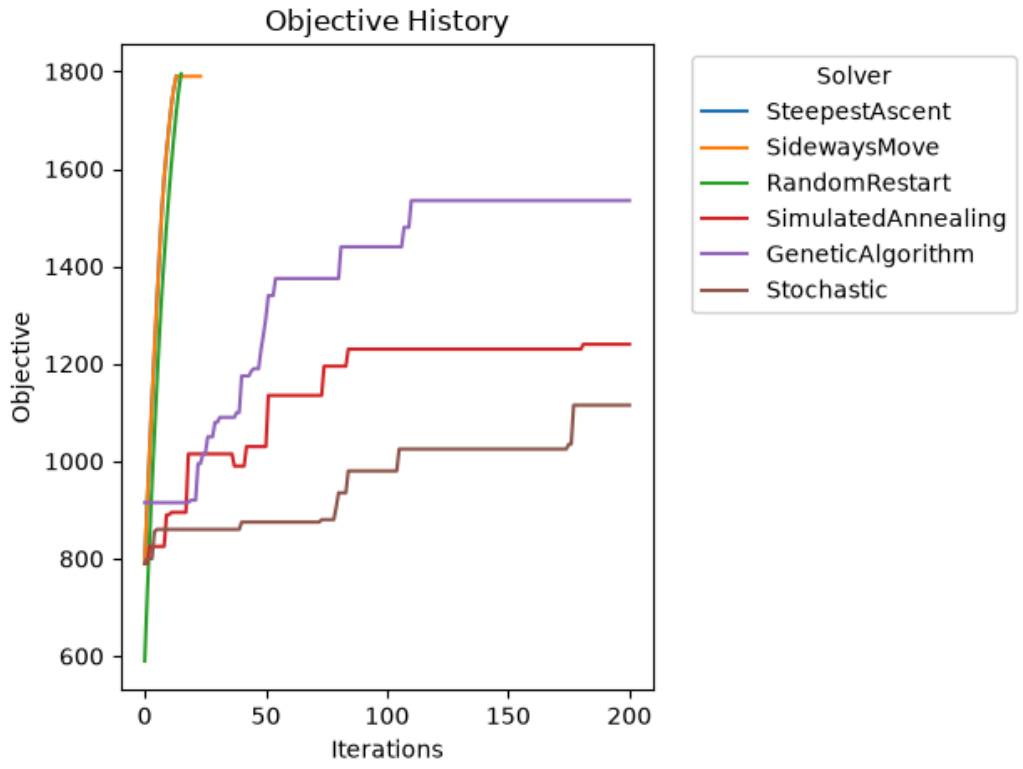
Visualization	

Genetic Algorithm

Information	**Value**
Runtime	0.450s
Total Iterations	200
Final Objective	1535.00
Visualization	

Perbandingan antara seluruh algoritma dapat dilihat pada tabel berikut ini.

Solver	Runtime (s)	Final Objective
Steepest Ascent Hill Climbing	70.800	1790
Stochastic Hill Climbing	0.020	1115
Sideways Move Hill	130.893	1790

Climbing		
Random Restart Hill Climbing	1333.034	1795
Simulated Annealing	0.038	1240
Genetic Algorithm	0.450	1535
Objective History		
		

Berdasarkan perbandingan antara keenam algoritma pencarian yang diuji, dapat dilihat kemampuan masing-masing algoritma mendekati global optima. Terlihat bahwa algoritma *genetic algorithm*, *simulated annealing*, dan *stochastic hill climbing* terjebak dalam local optima dan tidak mampu mendekati global optima. Algoritma *steepest ascent* dan *sideways move hill climbing* mampu mendekati global optima dengan jumlah iterasi yang lebih sedikit, tetapi juga masih gagal mencapai global optima. *Random restart hill climbing* menjadi satu-satunya algoritma yang mampu mencapai local optima paling tinggi, kemungkinan merupakan global optima. Hal ini disebabkan kemampuan untuk mencapai atau mendekati global optima ditentukan salah satunya oleh titik awal dimulainya proses pencarian (*initial state*). Algoritma *random restart hill climbing* memiliki kelebihan dibanding algoritma lainnya karena setiap *restart* dimulai dengan *initial state* yang

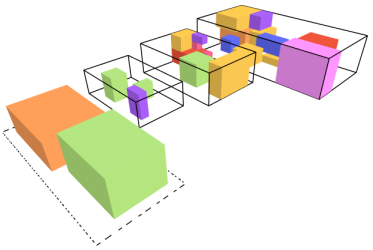
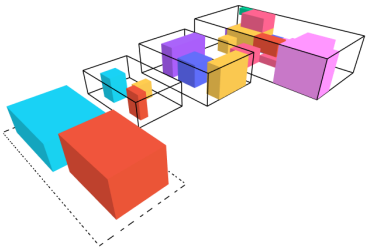
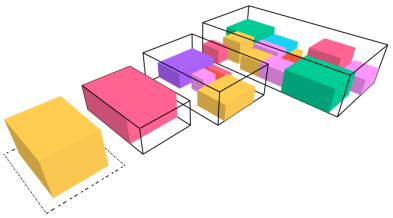
berbeda-beda melalui hasil randomisasi. Dengan demikian, algoritma ini memiliki peluang paling tinggi dibanding algoritma lainnya untuk dapat keluar dari local optima yang ditentukan oleh *initial state*.

Secara konsistensi pada masing-masing algoritma, dilihat dari kurva objektifnya terhadap iterasi, algoritma *steepest ascent, sideways move,* dan *random restart hill climbing* merupakan algoritma yang paling konsisten. Algoritma *steepest ascent* dan *sideways move hill climbing* secara khusus merupakan algoritma yang deterministik karena jalur yang diambil sudah pasti tergantung dari *initial state*-nya, sebab menggunakan *highest valued successor* sebagai pengganti *current state*-nya. Pada algoritma *random restart* sendiri, jalurnya ditentukan dengan hasil randomisasi *initial state* pada tiap *restart*. Untuk algoritma *stochastic hill climbing* dan *simulated annealing*, hasil pencariannya berbeda-beda, terutama pada iterasi awal, sebab algoritma ini menggunakan *random neighbor* sebagai pengganti *current state*. Pada algoritma *genetic algorithm*, kurva yang dicapai juga tidak konsisten karena pergantian *current state* bergantung pada hasil seleksi, *cross over*, dan mutasi.

Walaupun memiliki kemampuan mendekati global optima paling tinggi, *random restart hill climbing* memiliki kelemahan berupa kebutuhan kemampuan komputasi yang sangat tinggi, sehingga menyebabkan durasi pencarian yang jauh lebih lama dibanding algoritma lainnya. Bila dilihat pada tabel di atas, *random restart hill climbing* 10 kali lebih lama dari algoritma terlama kedua (*sideways move hill climbing*), dengan waktu pencariannya sebesar 1333s. Sementara itu, *stochastic hill climbing*, *simulated annealing*, dan *genetic algorithm* membutuhkan waktu di bawah satu detik untuk menyelesaikan proses pencariannya. Padahal, secara algoritma, ketika algoritma ini juga memiliki peluang untuk mampu melewati local optima dengan probabilitas tertentu. Pada *stochastic hill climbing* dan *simulated annealing*, seperti yang sudah dibuktikan pada subbab sebelumnya, dengan jumlah iterasi dan/atau jumlah pembangkitan *random neighbor* yang lebih tinggi, objektif algoritma juga mampu semakin mendekati global optima. Sementara itu, pada *genetic algorithm*, peningkatan jumlah populasi dan iterasi juga mampu semakin mendekati global optima. Oleh karena itu, untuk mengoptimalkan antara waktu pencarian dan nilai objektif, pendekatan *stochastic hill climbing* dan *simulated annealing* dengan iterasi dan/atau pembangkitan *successor* yang lebih tinggi, serta *genetic algorithm* dengan iterasi dan jumlah populasi yang lebih tinggi, secara berurutan, dapat menjadi pendekatan yang paling efisien.

3.4. Analisis Fungsi Objektif

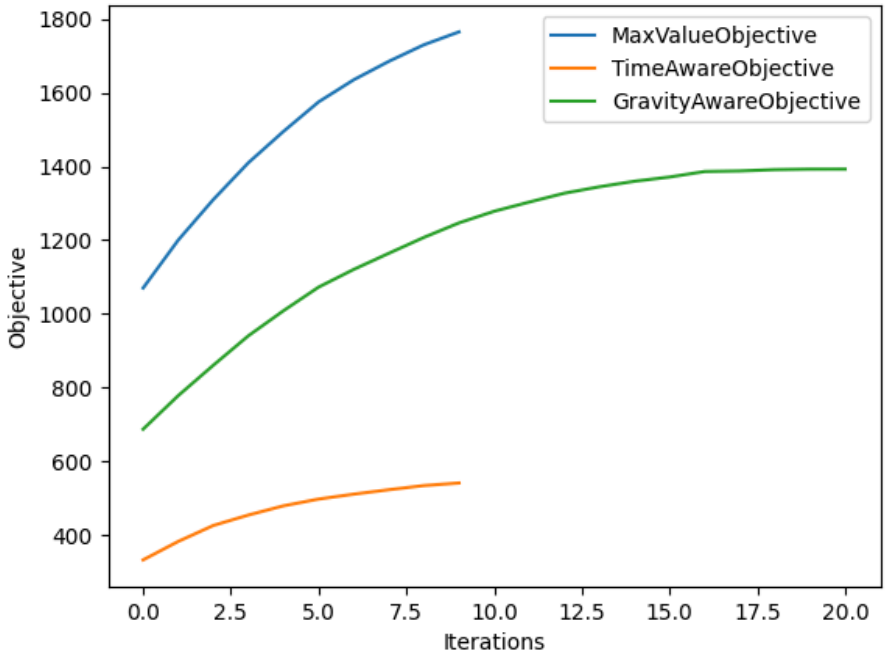
Bagian ini akan mendiskusikan pengaruh fungsi objektif pada solusi yang dihasilkan. Berdasarkan analisis perbandingan, performa jangka panjang ketiga algoritma *local search* serupa, dengan perbedaan paling mencolok adalah pada *runtime*. Oleh karena itu, akan diuji pengaruh fungsi objektif dengan menggunakan hanya *steepest ascent hill climbing*. Berikut hasil yang diperoleh.

Objective Function	Runtime	Iterations	Visualization
`MaxValueObjective`	55.047s	10	
`TimeAwareObjective`	63.143s	10	
`GravityAwareObjective`	135.437s	21	

Dari hasil di atas, terlihat bahwa `MaxValueObjective` and `TimeAwareObjective` menghasilkan solusi yang serupa dengan waktu komputasi yang serupa. Dari rumus mereka yang sudah mirip, tidak heran solusi yang diberikan juga. Selebihnya, *dataset* tidak memiliki variasi ETA yang sangat drastis sehingga tidak terlalu terlihat pengaruh `TimeAwareObjective`. Akan tetapi, fungsi objektif `GravityAwareObjective` terlihat sangat berpengaruh pada solusi. Fungsi objektif ini memprioritaskan paket-paket yang besar untuk ditaruh di bawah. Oleh karena itu, terlihat bahwa paket besar dengan berhasil dimasukkan dalam truk, sesuatu yang gagal dilakukan oleh fungsi objektif lainnya. Tetapi, keefektifan ini

datang dengan pengorbanan efisiensi, sebab `GravityAwareObjective` memiliki durasi *runtime* paling lambat dan banyak iterasi terbanyak.

Objective History



Grafik sejarah nilai objektif menunjukkan bahwa fungsi objektif *gravity aware* tidak hanya lebih mahal dihitung, tetapi memerlukan ketelitian lebih untuk mencapai suatu solusi optimum, sehingga algoritma memerlukan lebih banyak iterasi sebelum berhenti.

BAB IV
KESIMPULAN DAN SARAN

4.1. Kesimpulan

Enam algoritma yang digunakan pada percobaan ini memiliki kelebihan dan kekurangannya masing-masing berdasarkan kemampuan mencapai global optima, durasi pencarian, dan konsistensinya. Algoritma yang paling mampu mendekati global optima adalah *random restart hill climbing*, dengan jumlah iterasi yang juga lebih sedikit. Algoritma ini juga cenderung konsisten dalam jumlah iterasi yang sedikit, walaupun bila dibandingkan, algoritma *steepest ascent dan sideways move hill climbing* lebih konsisten tetapi lebih mudah terjebak di lokal optima. Walau demikian, algoritma *random restart hill climbing* membutuhkan kemampuan komputasi yang sangat besar dan durasi pencarian yang sangat lama, berkebalikan dengan *stochastic hill climbing*, *simulated annealing*, dan *genetic algorithm*. Akan tetapi, ketiga algoritma ini sangat tidak konsisten pada iterasi-iterasi awal, sehingga dibutuhkan jumlah iterasi yang tinggi untuk ketiga algoritma ini dan jumlah iterasi pembangkitan *random successor* yang tinggi untuk *stochastic hill climbing*. Pada *genetic algorithm*, jumlah populasi yang sesuai, tinggi tapi tidak terlalu tinggi dimana pada kasus ini sebanyak 6 populasi, juga mampu meningkatkan kemampuan *genetic algorithm* mendekati global optima. Oleh karena itu, pendekatan terbaik yang paling efisien secara waktu komputasi dan kemampuan mendekati global optima adalah *stochastic hill climbing* dan *simulated annealing* dengan jumlah iterasi yang tinggi dan iterasi pembangkitan *random successor* yang tinggi pada *stochastic hill climbing*.

4.2. Saran

Berdasarkan hasil eksperimen dan analisis yang telah dilakukan pada Tugas Besar 1 ini, terdapat beberapa saran untuk perbaikan pada percobaan selanjutnya. Pertama, pengujian dapat dilakukan kembali dengan variasi dataset yang lebih banyak dan kompleks untuk menguji keandalan program. Kedua, parameter untuk tiap algoritma masih dimasukkan secara manual oleh pengguna, sehingga akan lebih baik apabila program juga dapat melakukan hyperparameter tuning agar dapat menemukan konfigurasi yang paling cocok untuk masalah yang dimasukkan pengguna.

REFERENSI

Donaldson, T. (2019). *Notes on chapter 4: Beyond classical search*. Simon Fraser University. www.sfu.ca/~tjd/310summer2019/chp4_beyond_classical.html

GeeksforGeeks. (2026, June 19). *Problem solving in artificial intelligence*. www.geeksforgeeks.org/artificial-intelligence/problem-solving-in-artificial-intelligence/

Institut Teknologi Bandung. (2026). *Beyond classical search* [Lecture slides]. IF3170 Inteligensi Artifisial, Semester I TA 2026/2027.

Lemoine, O.-R. (2026). *Simulated annealing scientific cooling catalog*. Metaheuristics Platform. lemoine-or.github.io/MetaheuristicsPlatform/components/simulated-annealing-cooling-schedules.html

Russell, S. J., & Norvig, P. (2021). *Artificial intelligence: A modern approach* (4th ed., Global ed.). Pearson.

Goldberg, D. E. (1989). *Genetic algorithms in search, optimization, and machine learning*. Addison-Wesley.

LAMPIRAN

A. Pembagian Tugas Kelompok

NIM	Nama	Pembagian Tugas
10123053	Timothy Niels Ruslim	representasi persoalan, simulated annealing, laporan
10623061	Theodore Marvel Suranton	hill climbing, analisis perbandingan, laporan
10823501	Qodri Azkarayan	genetic algorithm, bonus objektif, laporan

B. Tautan Penting

GitHub	github.com/timoruslim/IF3170_G60_MoreArtificialThanIntelligence
YouTube	–

SURAT PERNYATAAN PENGGUNAAN AI

Dengan ini, kami:

Nama/NIM	:	Timothy Niels Ruslim (10123053), Theodore Marvel Suranton (10623061), Qodri Azkarayan (10823501)
Fakultas/Sekolah	:	Sekolah Teknik Elektro dan Informatika (STEI)
Kode Mata Kuliah	:	IF3170
Nama Mata Kuliah	:	Intelegensi Artifisial
Judul Tugas/Proposal	:	Tugas Besar 1

Menyatakan bahwa kami menggunakan AI dalam pengerjaan maupun penyusunan luaran tugas pada mata kuliah yang tertulis di atas.

Alat AI/kecerdasan buatan yang digunakan adalah:

Lingkup Pekerjaan	**Digunakan?**	**Tingkat Penggunaan**
Pemeriksaan Ejaan: tidak berlaku.	Tidak	1
Pembuatan Teks: tidak berlaku	Tidak	1
Bantuan Diskusi dalam Penyusunan Konten: tools seperti *Gemini*.	Ya	2
Pembuatan Gambar, Video, dan/atau Grafik: tidak berlaku.	Tidak	1
Bantuan Debugging dalam Penyusunan Kode: tools seperti *Gemini* dan *Claude*.	Ya	2

Kami juga menyatakan bahwa setiap penggunaan AI/kecerdasan buatan yang dilakukan pada mata kuliah tersebut telah dinyatakan di dalam surat ini.

Bandung, 18 Oktober 2026



Timothy Niels Ruslim
10123053